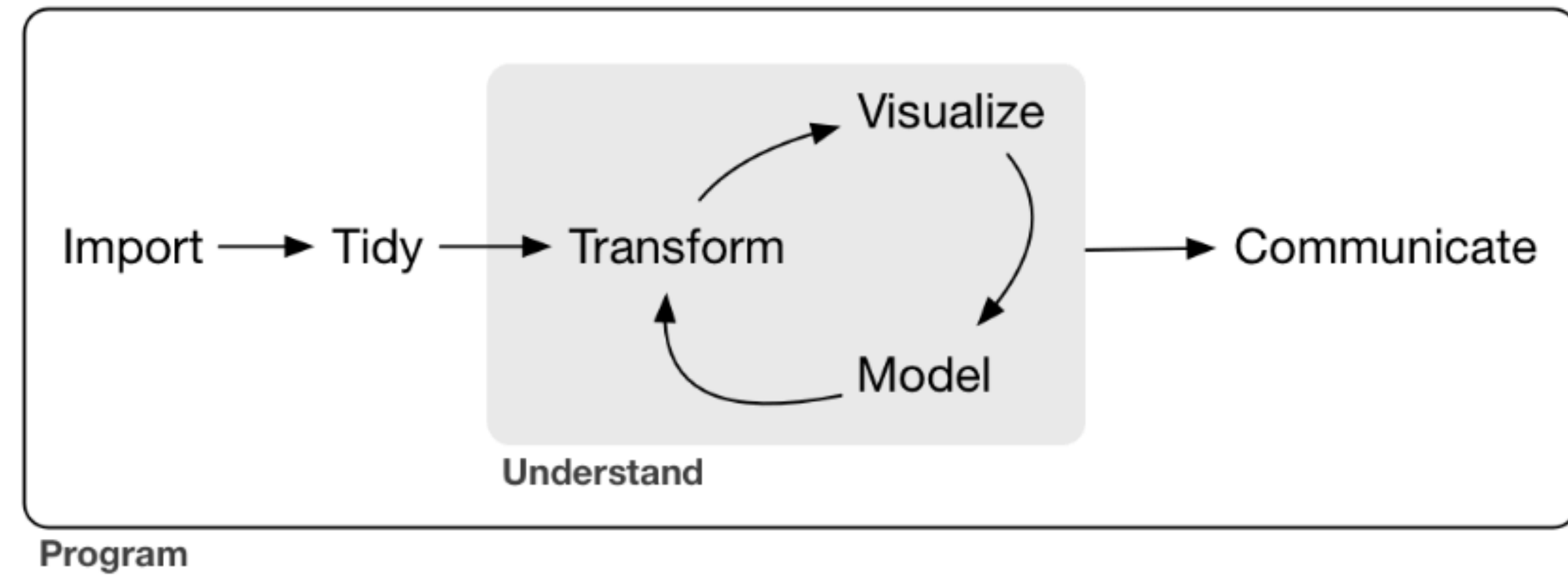




Minería de Datos

Alex Di Genova

07/04/2026

Programa

- Introducción a las técnicas principales en MD
- Data wrangling

Knowledge Discovery in Databases - KDD

Datos → información → conocimiento

- **Selección de datos**

Se eligen los datos relevantes desde una o varias fuentes.

Ejemplo: registros de ventas, datos clínicos, sensores, encuestas.

- **Limpieza y preprocesamiento**

Se corrigen errores, se eliminan duplicados, se manejan valores faltantes y ruido.

Esta etapa es clave, porque datos malos producen resultados malos.

- **Transformación de datos**

Los datos se preparan para el análisis.

Esto puede incluir normalización, reducción de dimensiones, agregación o creación de nuevas variables.

- **Minería de datos (Data Mining)**

Aquí se aplican métodos computacionales y estadísticos para identificar patrones.

- clasificación
- clustering
- reglas de asociación
- regresión
- detección de anomalías

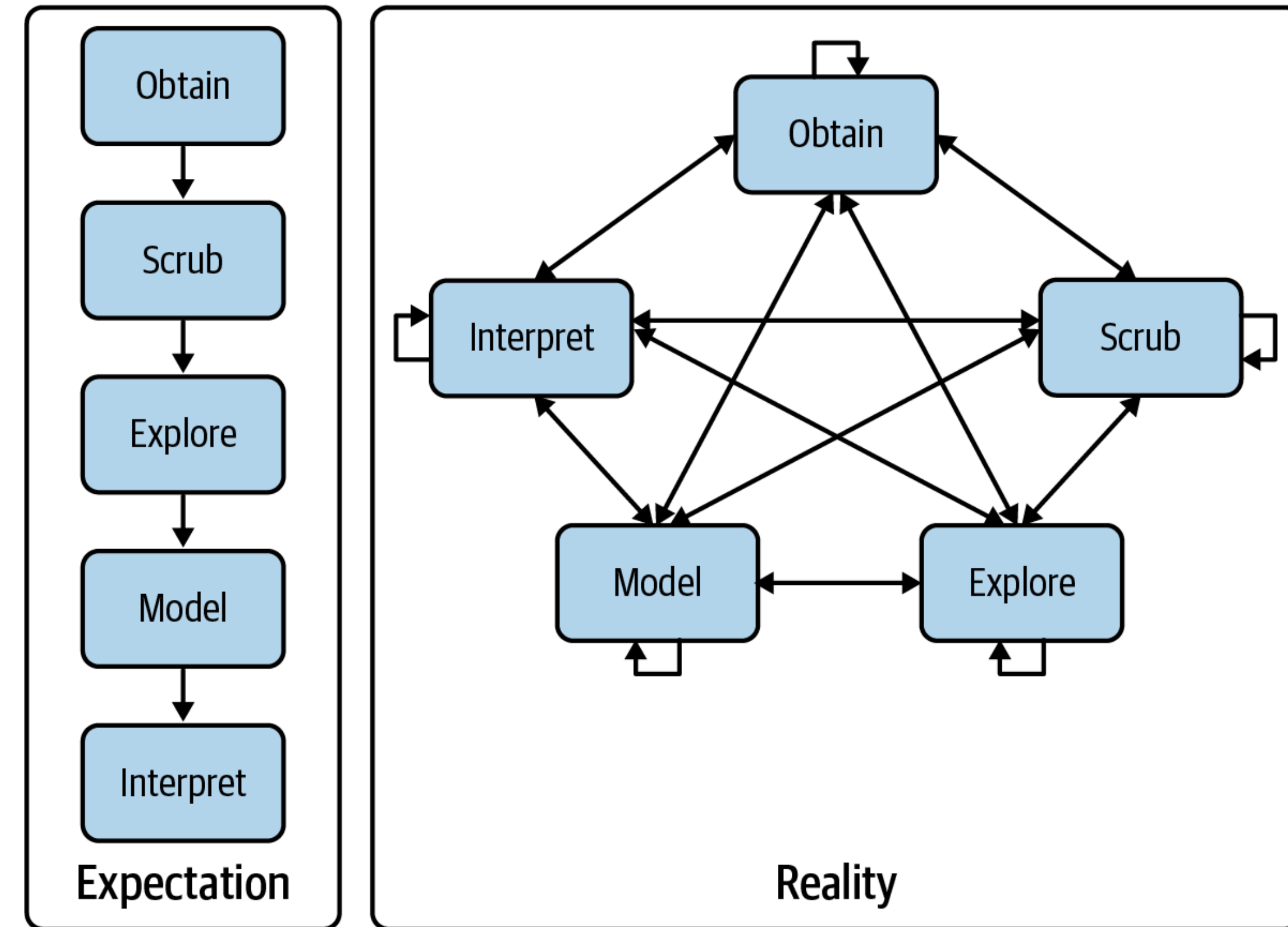
- **Evaluación e interpretación**

Se revisa si los patrones encontrados son realmente útiles, válidos e interesantes.

No todo patrón descubierto tiene valor práctico.

- **Presentación del conocimiento**

Los resultados se comunican de forma comprensible, por ejemplo, mediante gráficos, reglas, reportes o dashboards.



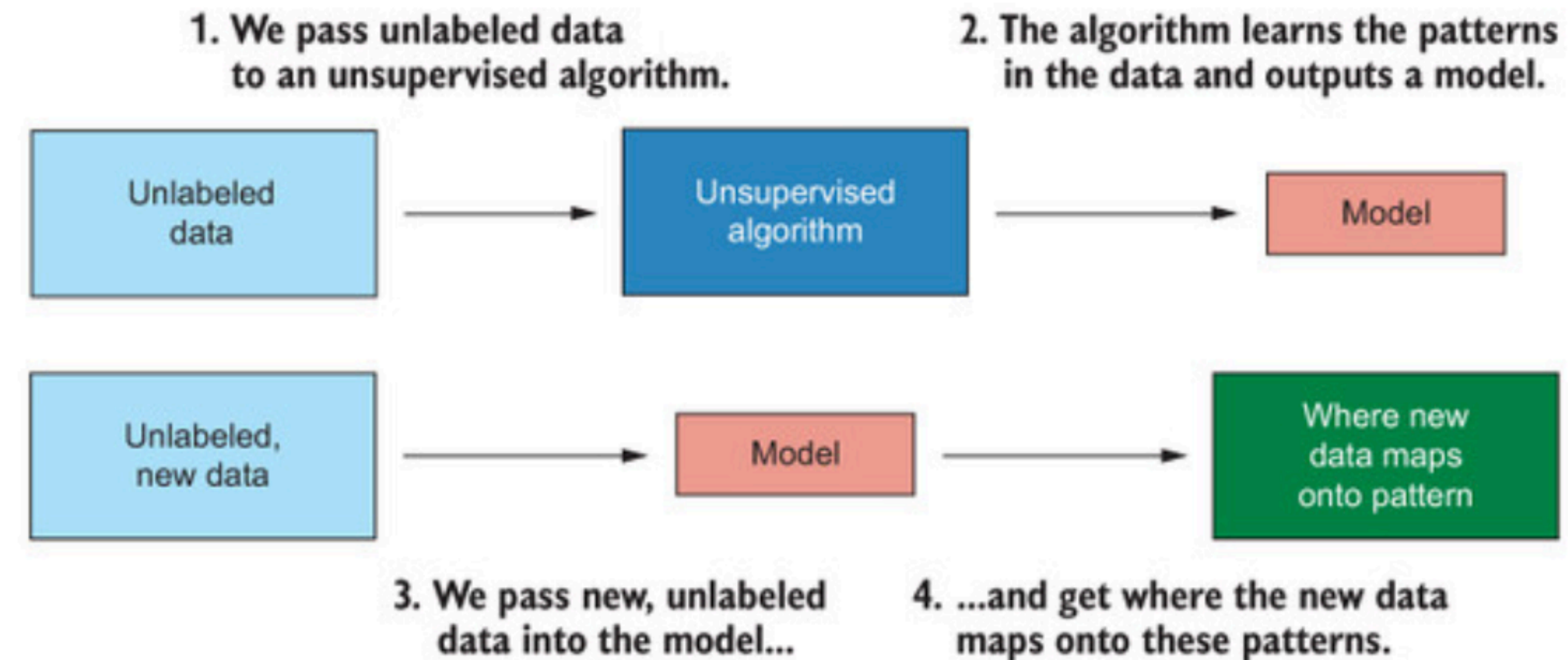
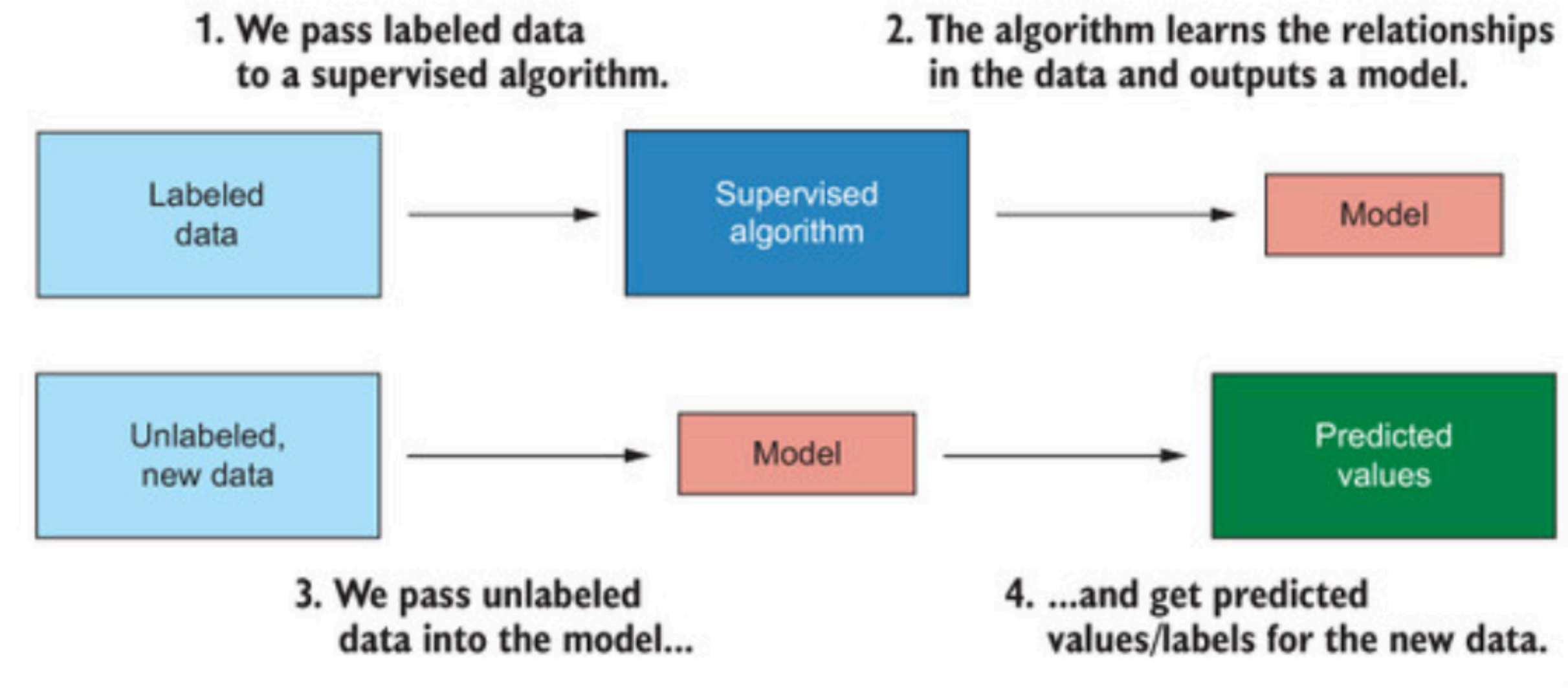
KDD es el proceso de identificar patrones válidos, nuevos, útiles y comprensibles en conjuntos de datos.

Técnicas principales en MD

Algoritmos de Minería de Datos

Clases

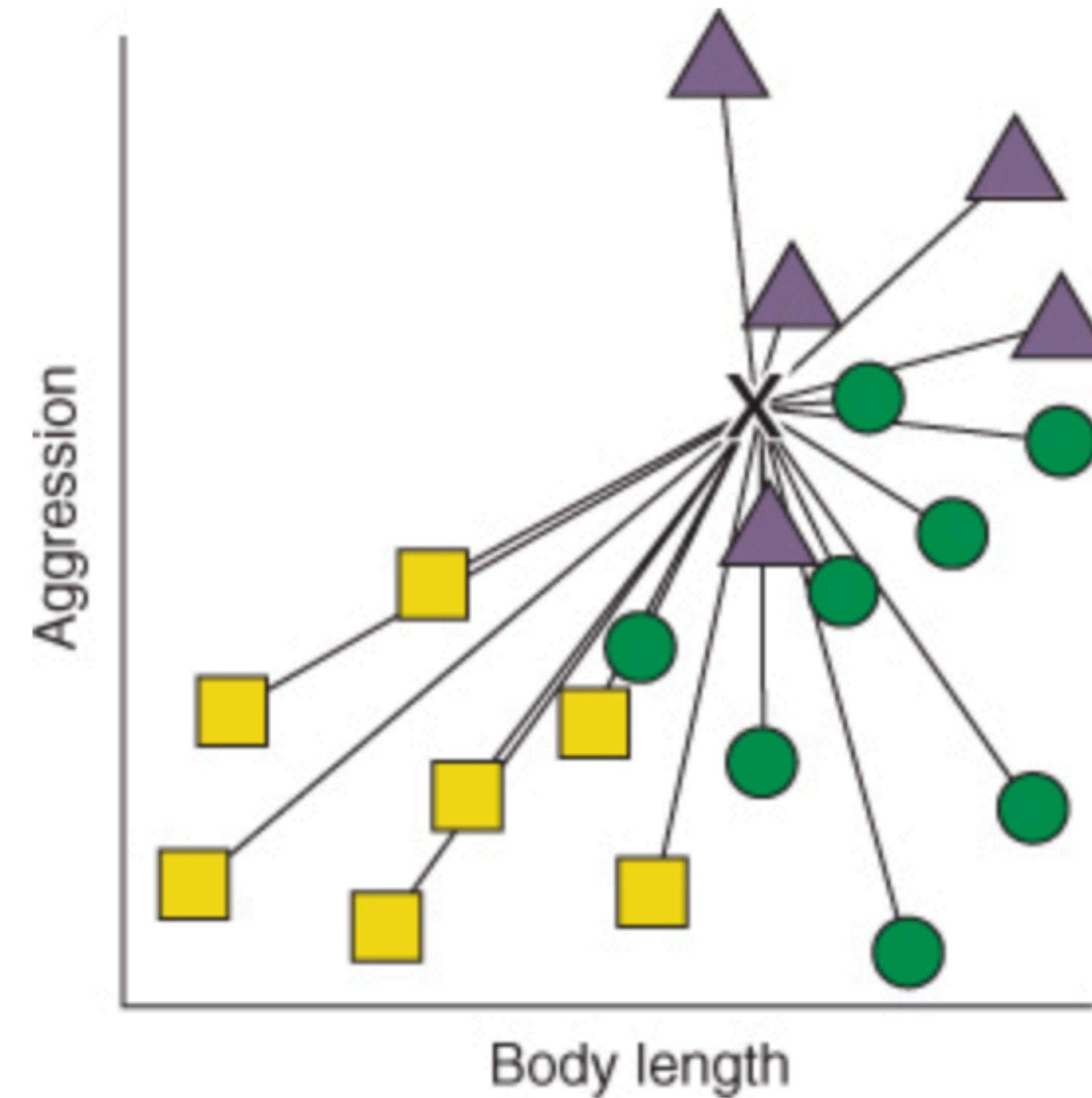
- Modelos estadísticos
- Supervisados
 - clasificación
 - Regresión
- No-supervisados
 - Reducción de dimensionalidad
 - Clustering
 - Analisis de asociación



Técnicas MD

Clasificación

- Aprende a separar entre clases a partir de ejemplos etiquetados.
- ¿Qué responde?
- Es spam o no spam?
- ¿Paciente con bajo o alto riesgo?
- ¿Cliente abandonara o permanecerá?



El algoritmo kNN calcula la distancia entre los casos sin etiqueta y todos los datos con etiqueta

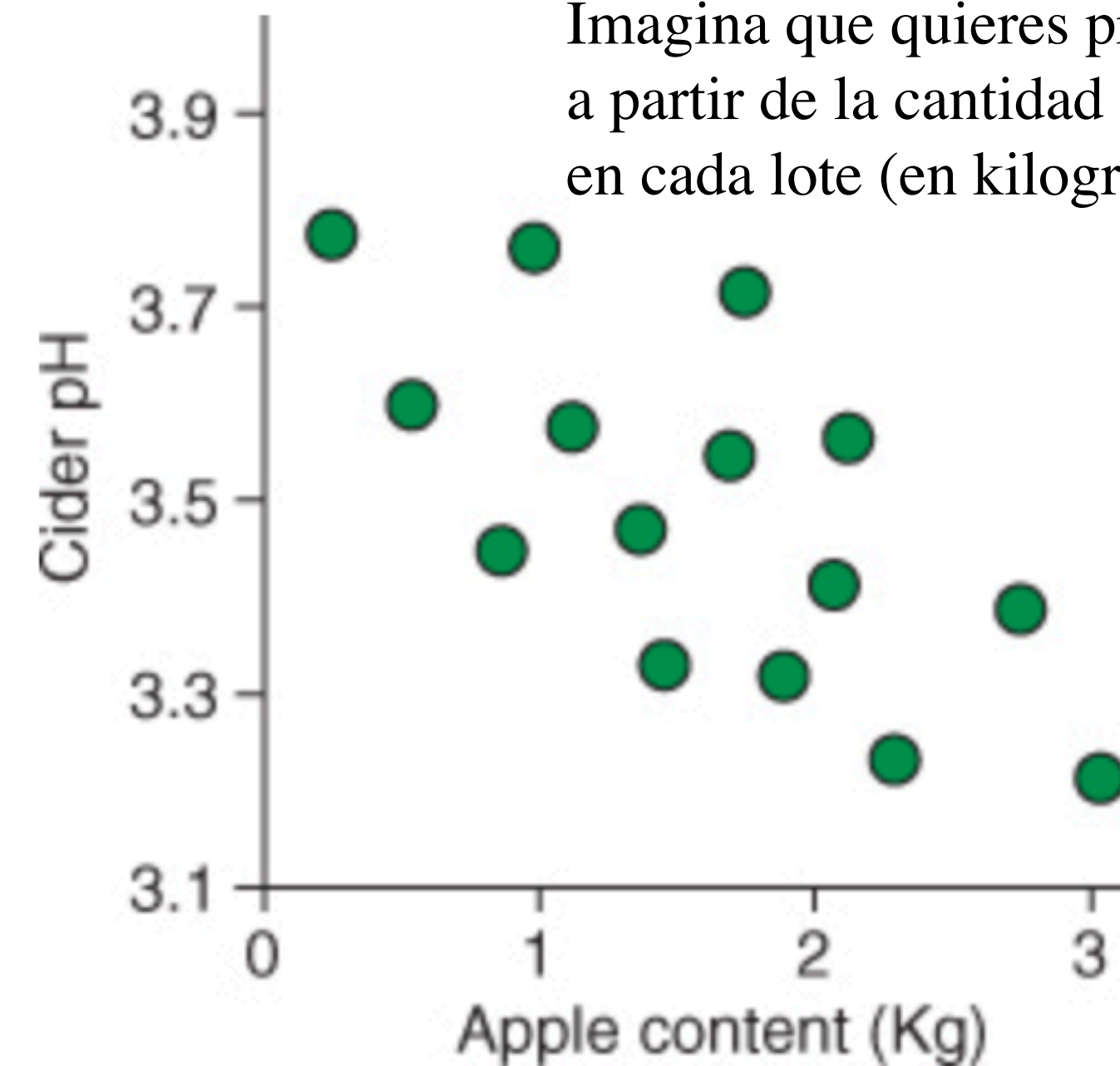
Dime con quién andas y te diré quién eres.

Técnicas MD

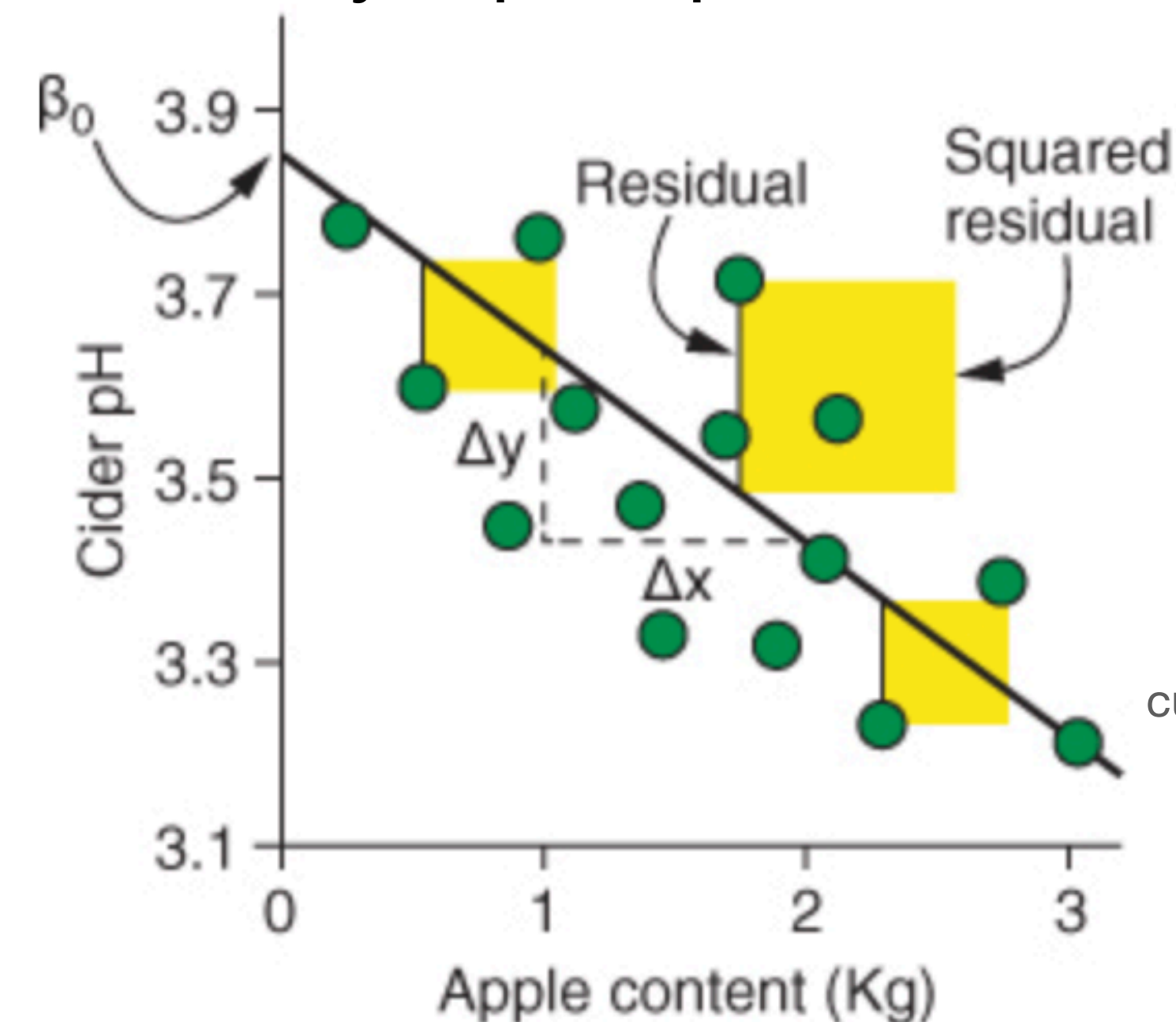
Regresión

- Modela una relación continua entre variables predictoras y una respuesta numérica.
- ¿Qué responde?
 - ¿Cuál será el precio de una vivienda?
 - ¿Cuántas ventas se esperan la próxima semana?
 - ¿Qué tiempo de recuperación tendrá un paciente?

Imagina que quieres predecir el pH de lotes de sidra a partir de la cantidad de manzana contenida en cada lote (en kilogramos).



$$y = \beta_0 + \beta_1 x_1 + \epsilon$$



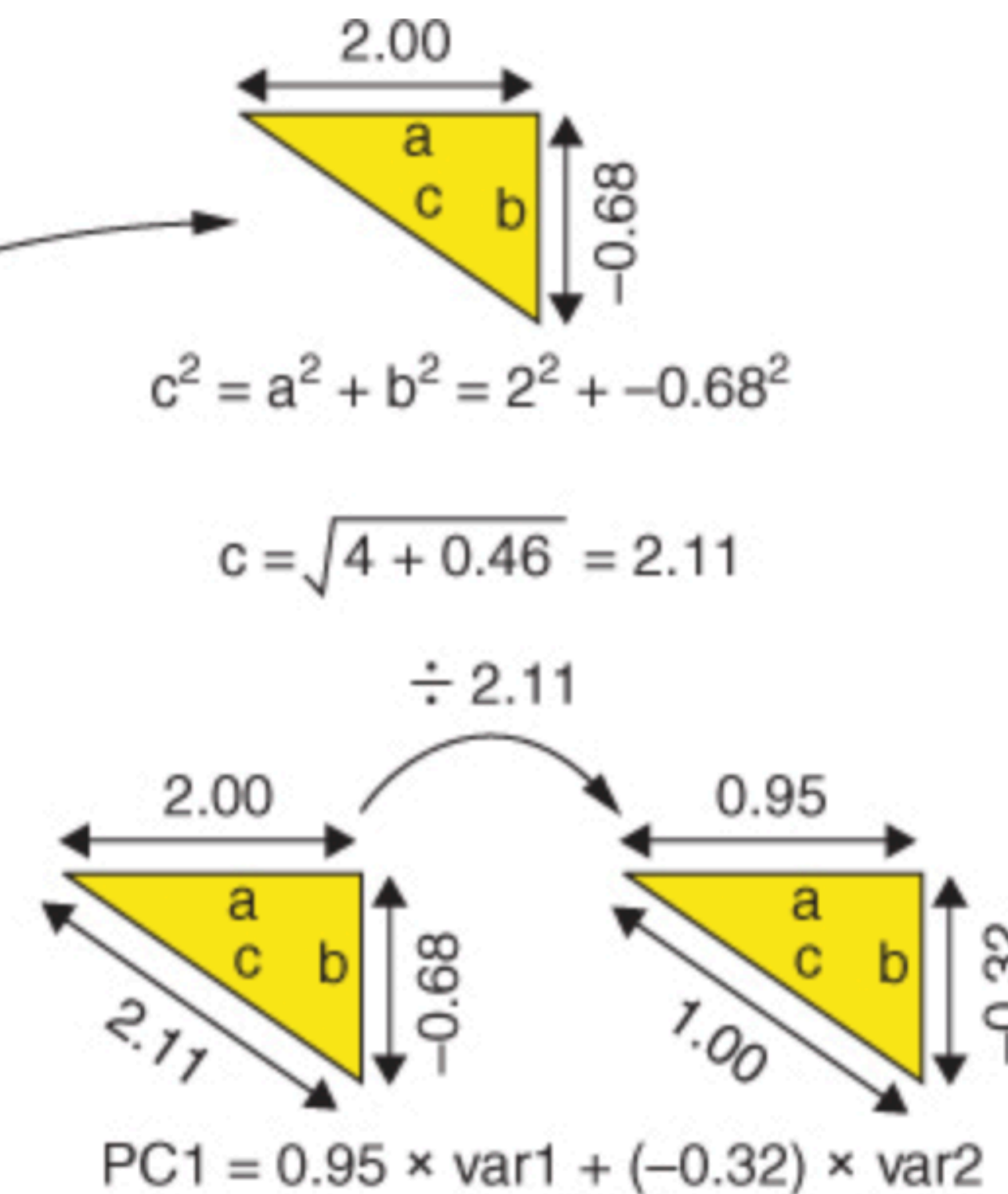
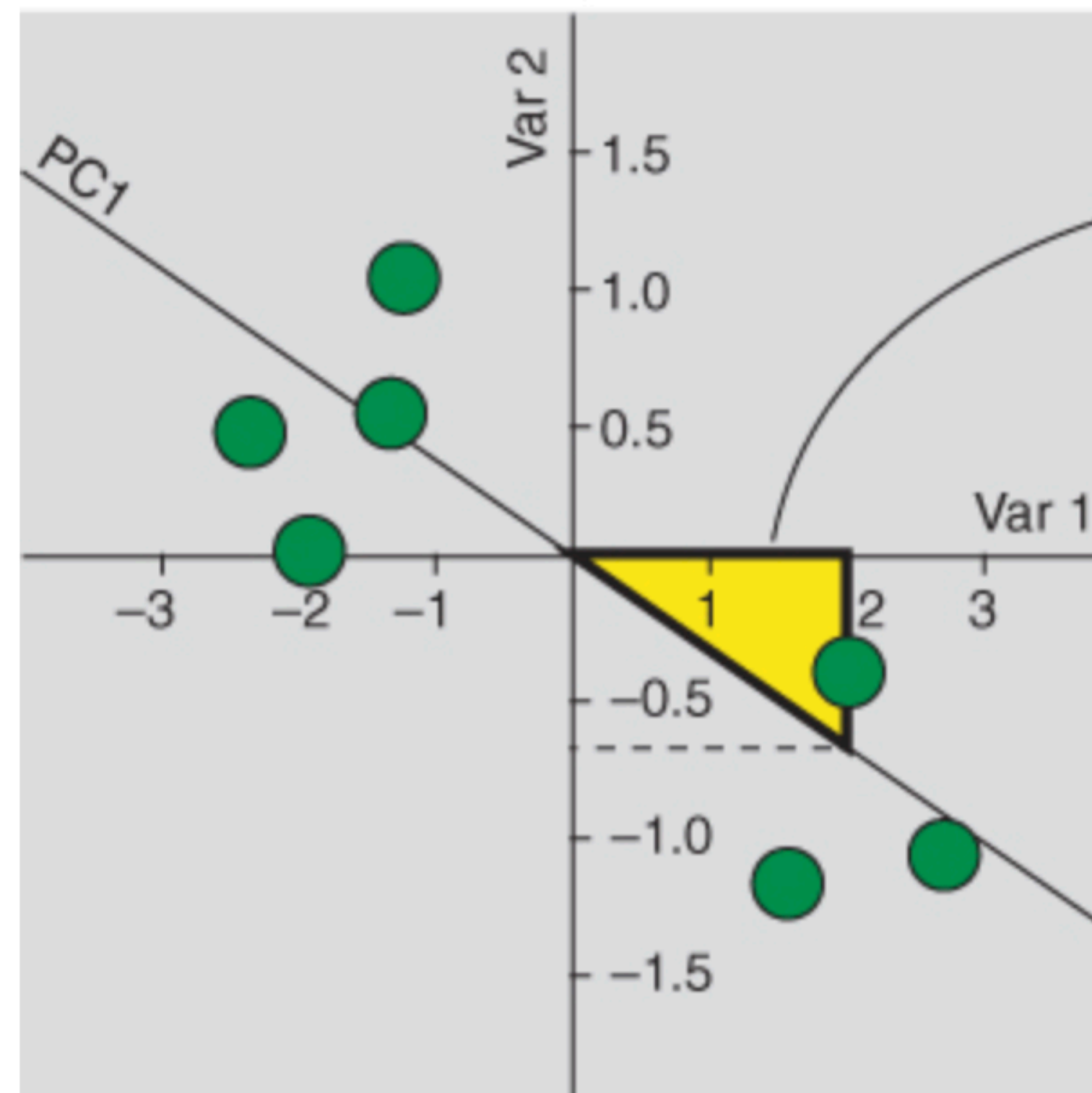
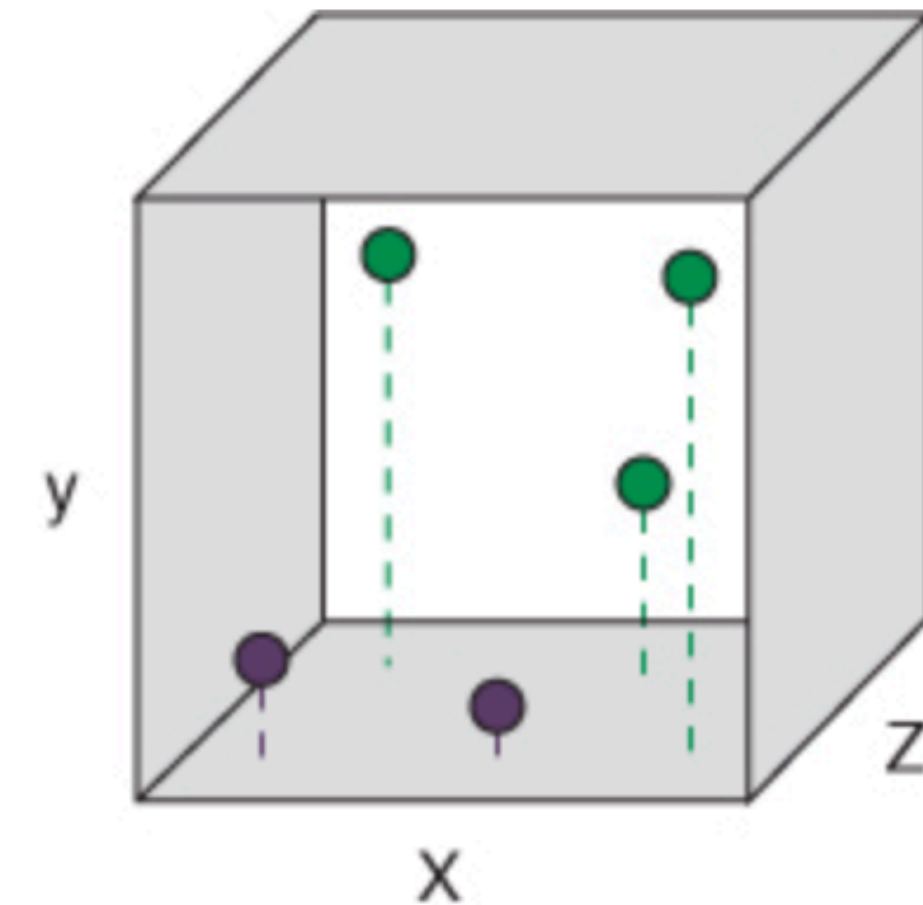
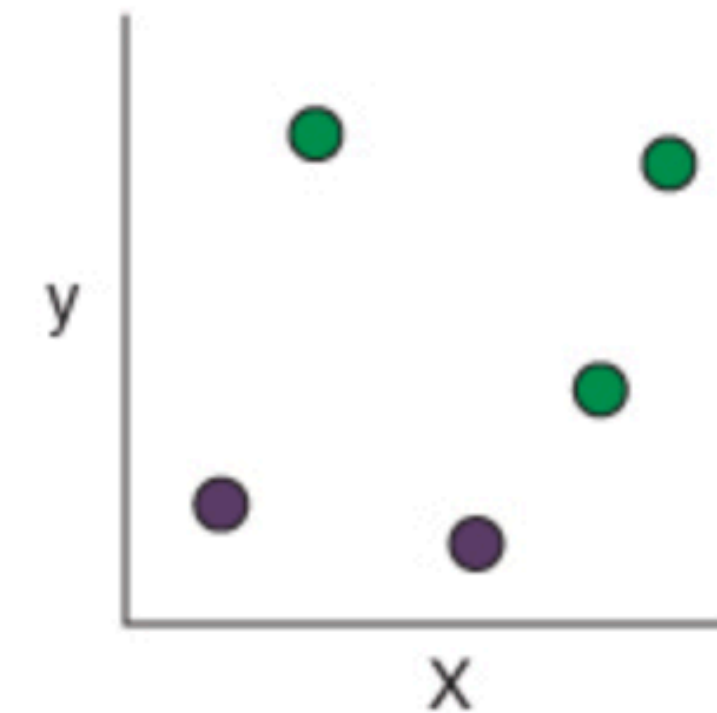
$$\begin{aligned}\beta_1 &= \frac{\Delta y}{\Delta x} \\ &= \frac{-0.105}{1} \\ &= -0.105\end{aligned}$$

Slope: como cambia Y cuando X incremente 1 unidad

Técnicas MD

Reduccion de dimensionalidad

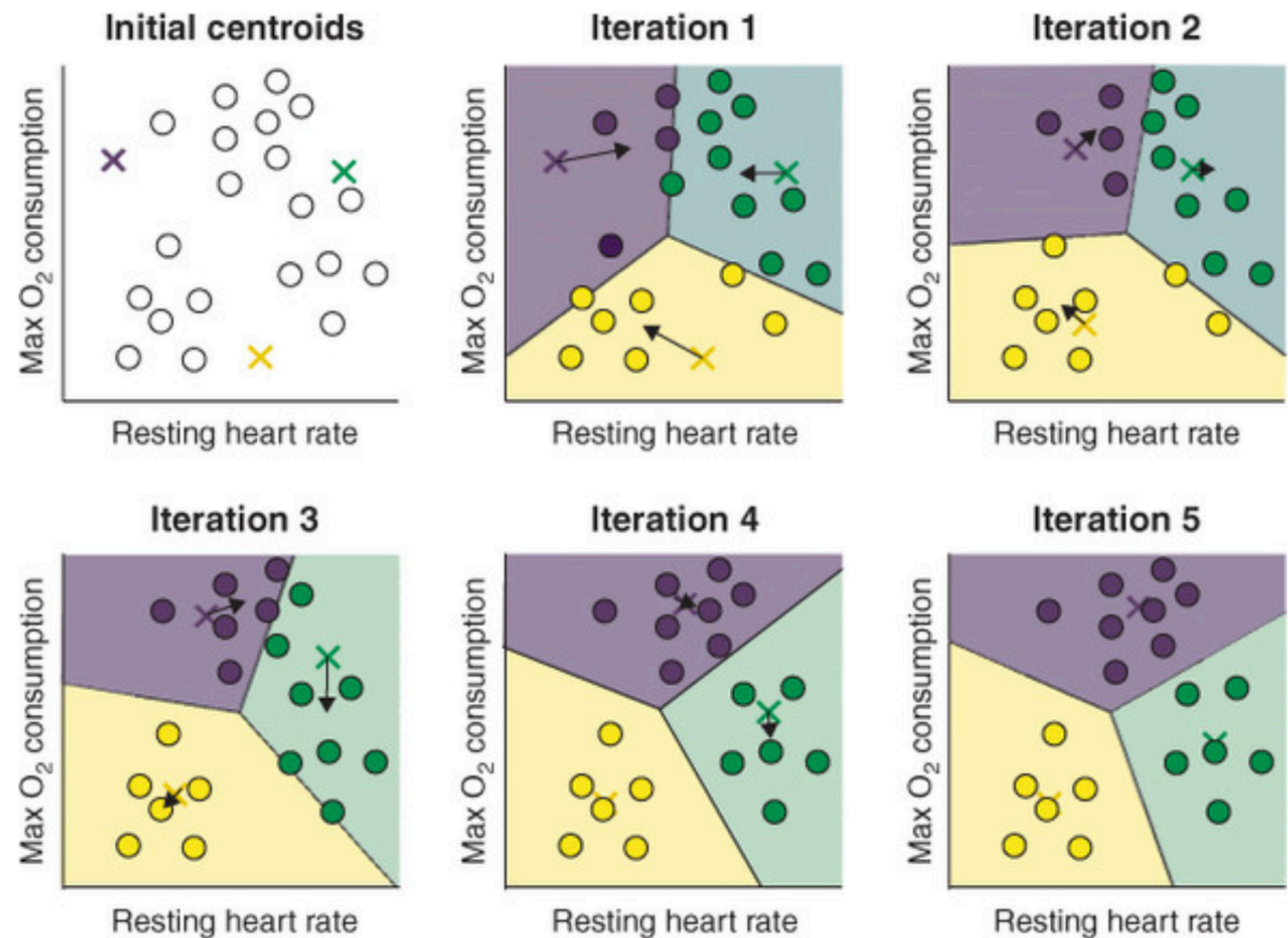
- Resume muchas variables en pocas componentes conservando estructura o variabilidad.
- ¿Qué responde?
- ¿Cómo visualizar datos de alta dimensión?
- ¿Qué variables sintetizan/explican mejor la información?
- ¿Cómo reducir ruido y redundancia?



Técnicas MD

Clustering

- Un clúster es un conjunto de casos que son más similares entre sí que a los casos de otros clústeres.
- ¿Qué responde?
- ¿Existen perfiles/grupos en los datos?
- ¿Qué clientes o pacientes son similares entre sí?
- ¿Cómo segmentar poblaciones heterogéneas?



5 iteraciones de k-means

Se generan aleatoriamente tres centros iniciales en el espacio de características. Los casos se asignan al clúster de su centro más cercano. En cada iteración, cada centro se desplaza hacia la media de los casos de su clúster

Técnicas MD

Análisis de asociación

- Descubre co-ocurrencias frecuentes y reglas del tipo $A \rightarrow B$ en datos transaccionales.
- ¿Qué responde?
 - ¿Qué elementos aparecen juntos con frecuencia?
 - ¿Qué combinaciones sirven para recomendar?
 - ¿Qué patrones de co-ocurrencia se repiten?

Base de transacciones

T1: {Pan, Leche}

T2: {Pan, Mantequilla}

T3: {Leche, Mantequilla}

T4: {Pan, Leche, Mantequilla}

T5: {Pan, Leche}

Objetivo:

Encontrar conjuntos frecuentes de

Soporte mínimo = 2 transacciones.

Idea central:

- 1) generar candidatos
- 2) contar su soporte
- 3) eliminar los no frecuentes
- 4) repetir con conjuntos más grandes

Resultado final

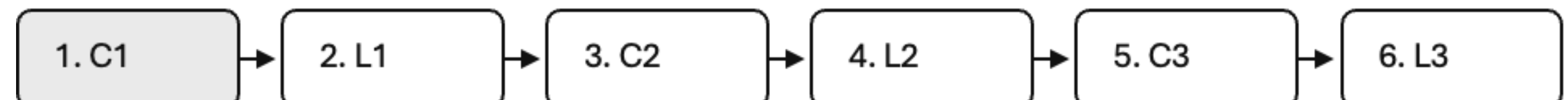
Conjuntos frecuentes encontrados:

L1 = {Pan}, {Leche}, {Mantequilla}

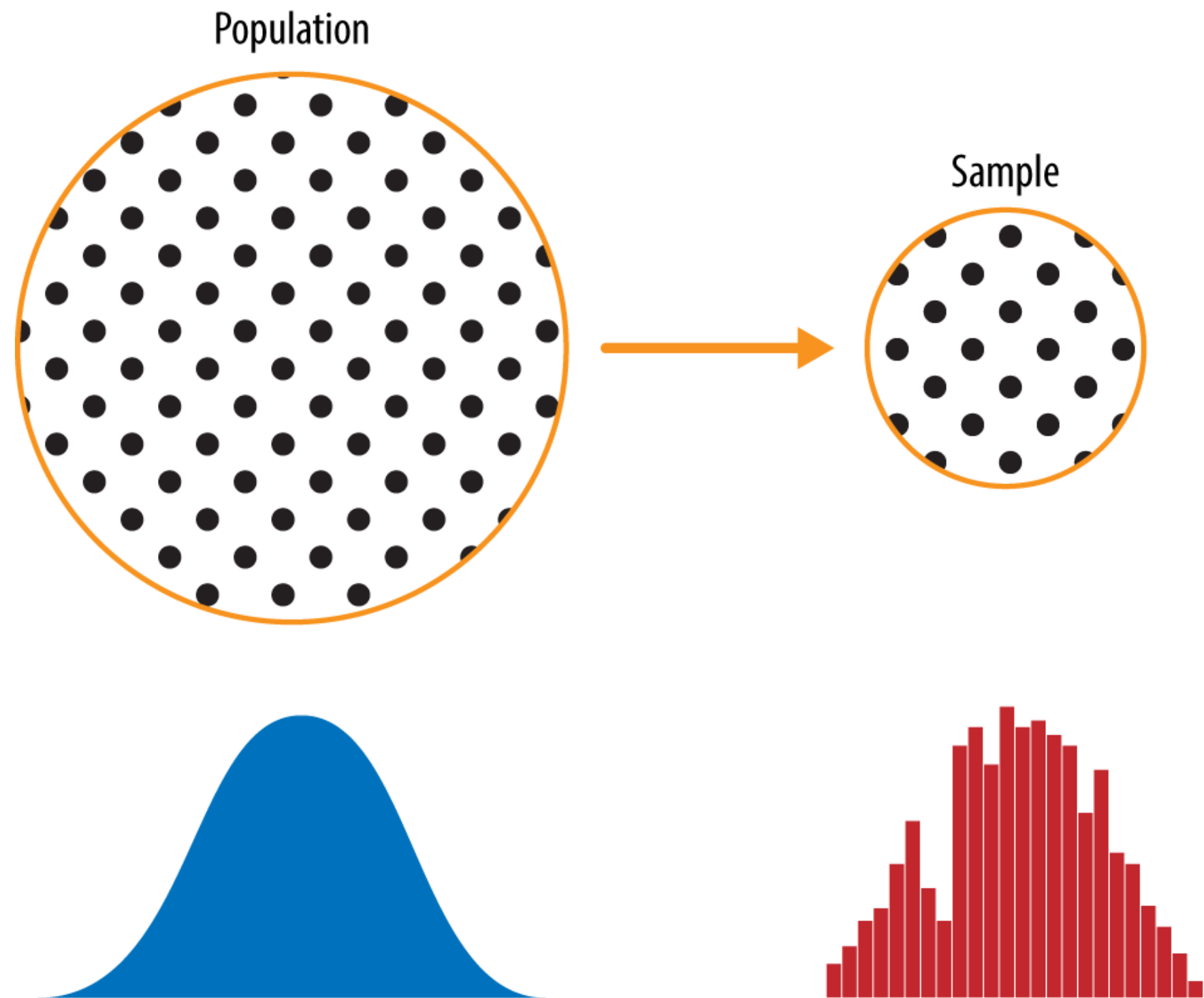
L2 = {Pan, Leche}, {Pan, Mantequilla}, {Leche, Mantequilla}

L3 = $\emptyset$

Apriori funciona porque si un conjunto NO es frecuente, ninguno de sus superconjuntos puede serlo.



Métodos estadísticos



3. Comparar medias entre tres o más grupos independientes

Pregunta

- ¿Existen diferencias de medias entre varios grupos?
- Ejemplo: ¿el rendimiento difiere entre 3 métodos de enseñanza?

Test

- **ANOVA de una vía**
- Si no hay normalidad: **Kruskal–Wallis**

1. Comparar una media con un valor de referencia

Pregunta

- ¿La media de una variable es distinta de un valor conocido?
- Ejemplo: ¿el promedio de presión arterial es distinto de 120?

Test

- **t de una muestra**
- Si no se cumple normalidad y la muestra es pequeña: **Wilcoxon de una muestra**

2. Comparar medias entre dos grupos independientes

Pregunta

- ¿El promedio de una variable difiere entre dos grupos?
- Ejemplo: ¿la nota promedio difiere entre hombres y mujeres?

Test

- **t de Student para muestras independientes**
- Si las varianzas no son iguales: **t de Welch**
- Si no hay normalidad: **Mann–Whitney U**

4. Modelar efecto de una variable sobre otra numérica

Pregunta

- ¿X tiene efecto sobre Y continua?
- Ejemplo: ¿la edad predice presión arterial?

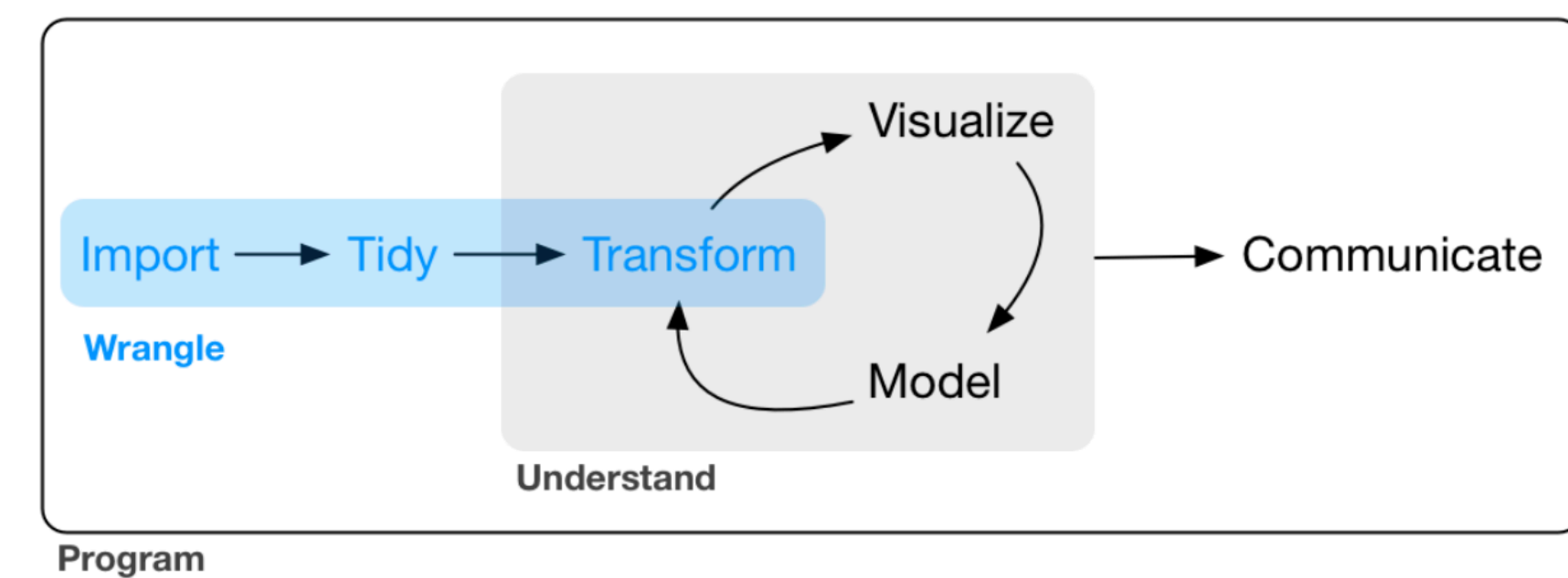
Test / enfoque

- **Regresión lineal**

Data wrangling

Tidyverse

Wrangle



- Getting your data in a helpful form for visualization and modeling

- data.frames and Tibbles

- Data import with readr

- read plain-text rectangular files

- From a file.txt to a data frame

- Common functions

- read_csv(), read_csv_delim(), read_csv_table()

```
### readr
```

```
`{r readdata}
```

```
d = read_csv("../data/worldcitiespop.csv.gz")
```

```
`{ }
```

Rows: 3173958 Columns: 7 — Column specification

Delimiter: " "

Rows: 3,173,958

Columns: 7

\$ Country <chr> "ad", "ad", "ad", "ad", "ad", "ad", "

\$ City <chr> "aixas", "aixirivali", "aixirivall",

\$ AccentCity <chr> "Aixàs", "Aixirivali", "Aixirivall",

\$ Region <chr> "06", "06", "06", "06", "06", "07", "

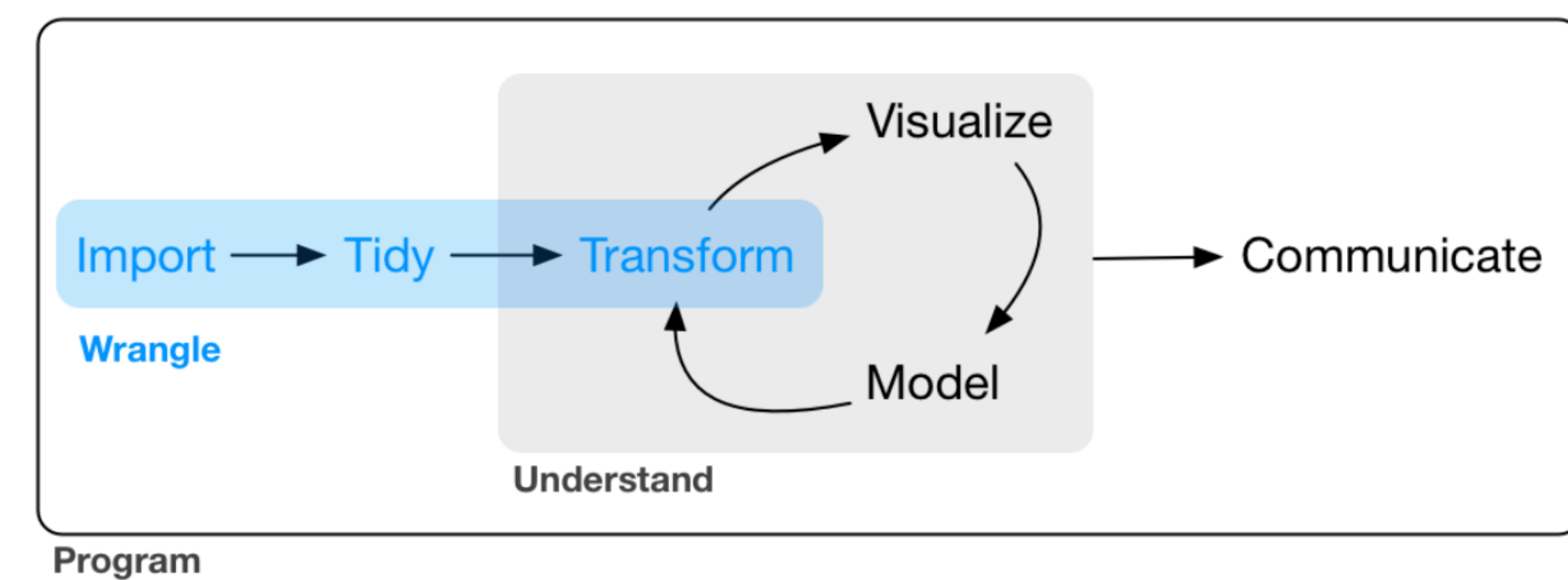
\$ Population <dbl> NA, NA, NA, NA, NA, NA, 20430, NA, NA

\$ Latitude <dbl> 42.48333, 42.46667, 42.46667, 42.4666

\$ Longitude <dbl> 1.466667, 1.500000, 1.500000, 1.500000

Tidyverse

Readr



```
read_csv("The first line of metadata
The second line of metadata
x,y,z
1,2,3", skip = 2)
```

```
read_csv("# A comment I want to skip
x,y,z
1,2,3", comment = "#")
```

```
read_csv("1,2,3\n4,5,6", col_names = c("x", "y", "z"))
```

- `parse_*()` functions:

- From a character vector to a specialized vector like logical or date.

- `parse_logical()`, `parse_date()`

```
```{r n}
parse_number("$100")
parse_number("20%")
parse_number("It cost $123.45")
```
```

```
[1] 100
[1] 20
[1] 123.45
```

```
```{r fechas}
parse_date("01/02/15", "%m/%d/%Y")
parse_date("01/02/15", "%d/%m/%Y")
parse_date("01/02/15", "%Y/%m/%d")
parse_date("12 Enero 2015", "%d %B %Y")
```
```

How readr automatically guesses

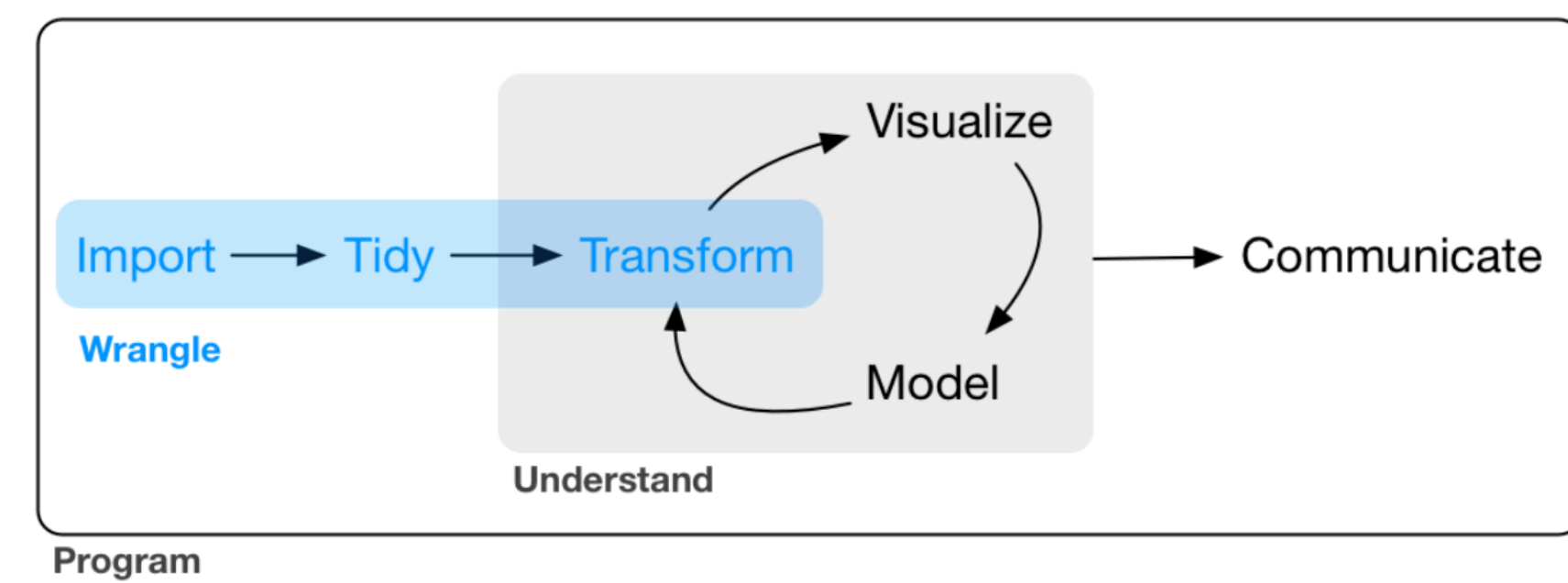
```
[1] "2015-01-02"
[1] "2015-02-01"
[1] "2001-02-15"
[1] "2015-01-12"
```

```
```{r guess}
guess_parser("2010-10-01")
guess_parser("15:01")
guess_parser(c("TRUE", "FALSE"))
guess_parser(c("1", "5", "9"))
guess_parser(c("12,352,561"))
```
```

```
[1] "date"
[1] "time"
[1] "logical"
[1] "double"
[1] "number"
```

Tidyverse

Readr



- `write_csv()` and `write_tsv()`
 - `write_csv(challenge, "challenge.csv")`
- `write_rds()` and `read_rds()`, store data in R's custom binary format called RDS
 - `write_rds(challenge, "challenge.rds")`
 - `read_rds("challenge.rds")`

Tidyverse

Tidy

Journal of Statistical Software

| | treatmenta | treatmentb |
|--------------|------------|------------|
| John Smith | — | 2 |
| Jane Doe | 16 | 11 |
| Mary Johnson | 3 | 1 |

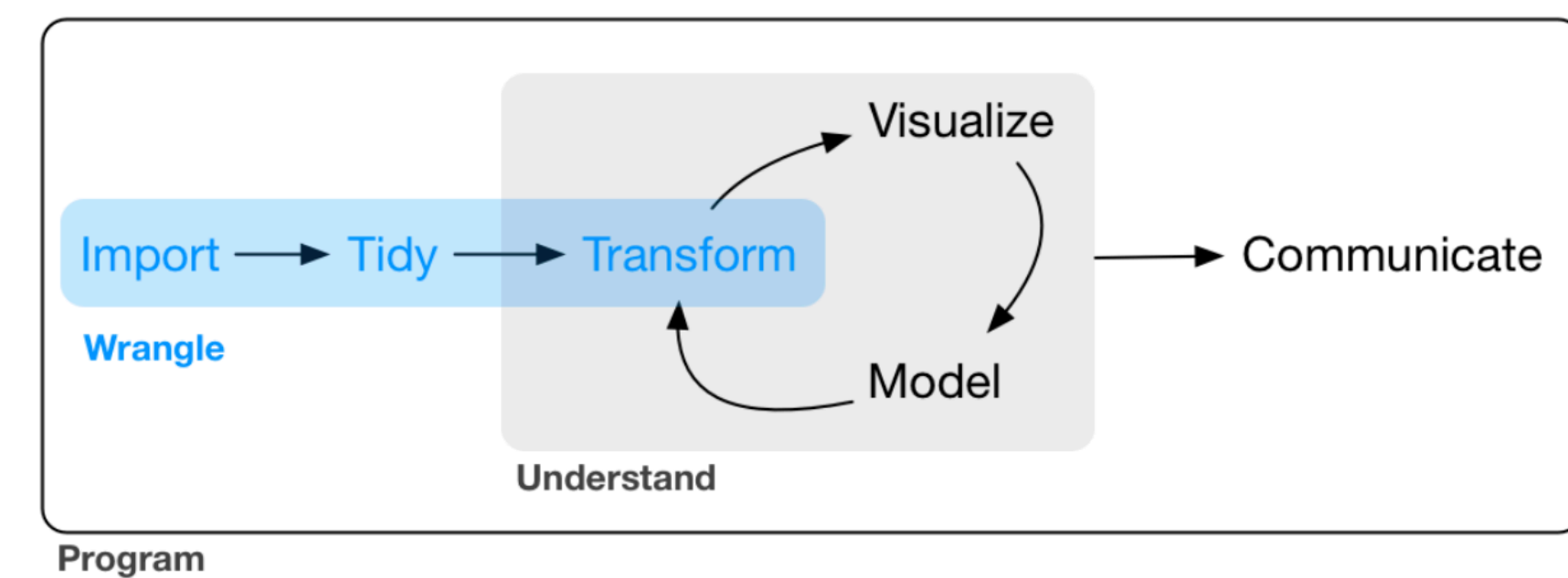
Table 1: Typical presentation dataset.

| | John Smith | Jane Doe | Mary Johnson |
|------------|------------|----------|--------------|
| treatmenta | — | 16 | 3 |
| treatmentb | 2 | 11 | 1 |

Table 2: The same data as in Table 1 but structured differently.

Tidy data

1. Each variable forms a column.
2. Each observation forms a row.
3. Each type of observational unit forms a table.



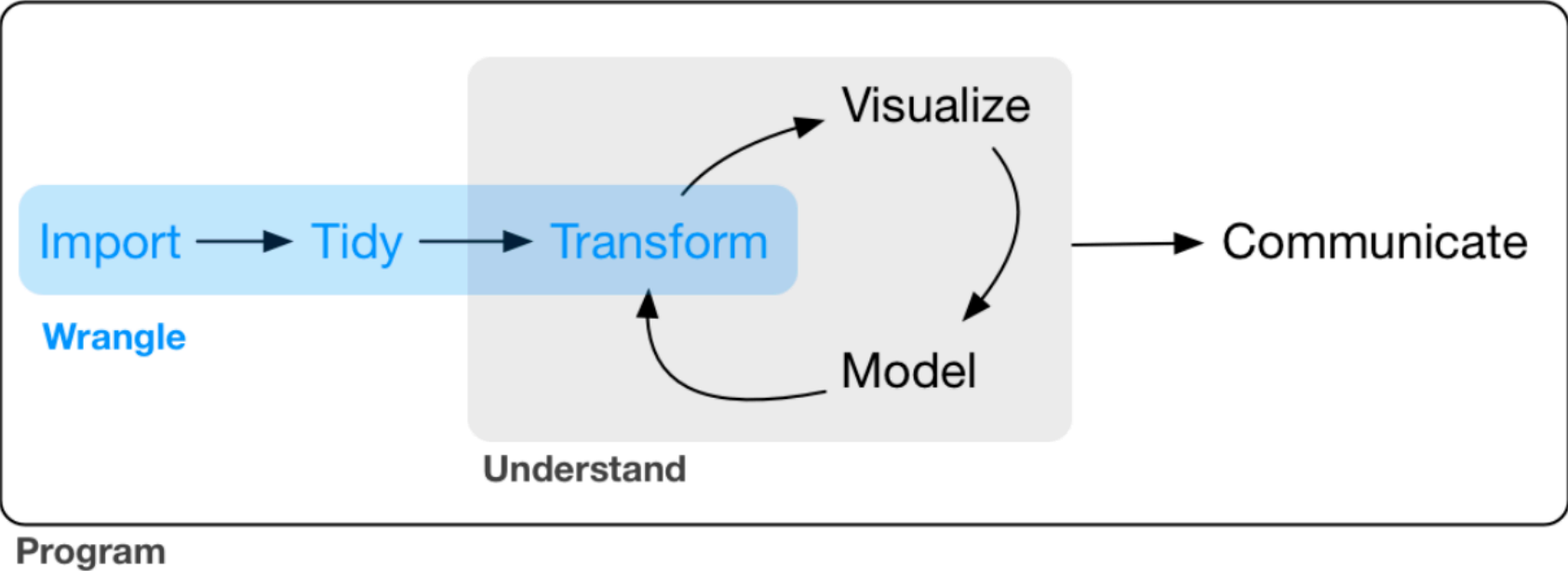
Tidy Data

| person | treatment | result |
|--------------|-----------|--------|
| John Smith | a | — |
| Jane Doe | a | 16 |
| Mary Johnson | a | 3 |
| John Smith | b | 2 |
| Jane Doe | b | 11 |
| Mary Johnson | b | 1 |

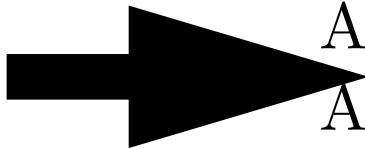
- Column headers are values, not variable names.
- Multiple variables are stored in one column.
- Variables are stored in both rows and columns.
- Multiple types of observational units are stored in the same table.
- A single observational unit is stored in multiple tables.

Tidyverse

Tidy



| religion | <\$10k | \$10–20k | \$20–30k | \$30–40k | \$40–50k | \$50–75k |
|-------------------------|--------|----------|----------|----------|----------|----------|
| Agnostic | 27 | 34 | 60 | 81 | 76 | 137 |
| Atheist | 12 | 27 | 37 | 52 | 35 | 70 |
| Buddhist | 27 | 21 | 30 | 34 | 33 | 58 |
| Catholic | 418 | 617 | 732 | 670 | 638 | 1116 |
| Don’t know/refused | 15 | 14 | 15 | 11 | 10 | 35 |
| Evangelical Prot | 575 | 869 | 1064 | 982 | 881 | 1486 |
| Hindu | 1 | 9 | 7 | 9 | 11 | 34 |
| Historically Black Prot | 228 | 244 | 236 | 238 | 197 | 223 |
| Jehovah’s Witness | 20 | 27 | 24 | 24 | 21 | 30 |
| Jewish | 19 | 19 | 25 | 25 | 30 | 95 |



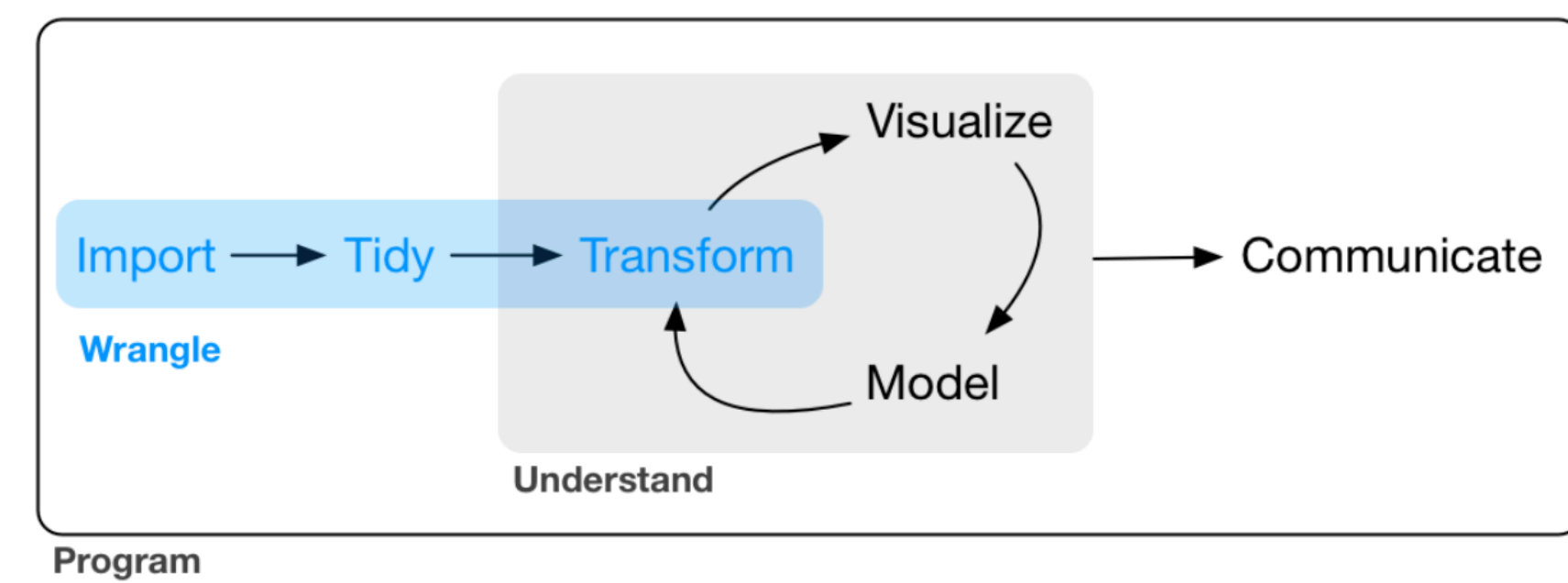
| religion | income | freq |
|----------|--------------------|------|
| Agnostic | <\$10k | 27 |
| Agnostic | \$10–20k | 34 |
| Agnostic | \$20–30k | 60 |
| Agnostic | \$30–40k | 81 |
| Agnostic | \$40–50k | 76 |
| Agnostic | \$50–75k | 137 |
| Agnostic | \$75–100k | 122 |
| Agnostic | \$100–150k | 109 |
| Agnostic | >150k | 84 |
| Agnostic | Don’t know/refused | 96 |

| id | year | month | element | d1 | d2 | d3 | d4 | d5 | d6 | d7 | d8 |
|---------|------|-------|---------|----|------|------|----|------|----|----|----|
| MX17004 | 2010 | 1 | tmax | — | — | — | — | — | — | — | — |
| MX17004 | 2010 | 1 | tmin | — | — | — | — | — | — | — | — |
| MX17004 | 2010 | 2 | tmax | — | 27.3 | 24.1 | — | — | — | — | — |
| MX17004 | 2010 | 2 | tmin | — | 14.4 | 14.4 | — | — | — | — | — |
| MX17004 | 2010 | 3 | tmax | — | — | — | — | 32.1 | — | — | — |
| MX17004 | 2010 | 3 | tmin | — | — | — | — | 14.2 | — | — | — |
| MX17004 | 2010 | 4 | tmax | — | — | — | — | — | — | — | — |
| MX17004 | 2010 | 4 | tmin | — | — | — | — | — | — | — | — |
| MX17004 | 2010 | 5 | tmax | — | — | — | — | — | — | — | — |
| MX17004 | 2010 | 5 | tmin | — | — | — | — | — | — | — | — |

| id | date | tmax | tmin |
|---------|------------|------|------|
| MX17004 | 2010-01-30 | 27.8 | 14.5 |
| MX17004 | 2010-02-02 | 27.3 | 14.4 |
| MX17004 | 2010-02-03 | 24.1 | 14.4 |
| MX17004 | 2010-02-11 | 29.7 | 13.4 |
| MX17004 | 2010-02-23 | 29.9 | 10.7 |
| MX17004 | 2010-03-05 | 32.1 | 14.2 |
| MX17004 | 2010-03-10 | 34.5 | 16.8 |
| MX17004 | 2010-03-16 | 31.1 | 17.6 |
| MX17004 | 2010-04-27 | 36.3 | 16.7 |
| MX17004 | 2010-05-27 | 33.2 | 18.2 |

Tidyverse

Tidy



| country | year | cases | population |
|-------------|------|-------|------------|
| Afghanistan | 1999 | 75 | 19987071 |
| Afghanistan | 2000 | 866 | 20595360 |
| Brazil | 1999 | 3737 | 17206362 |
| Brazil | 2000 | 8488 | 174604898 |
| China | 1999 | 21258 | 127215272 |
| China | 2000 | 21766 | 128042583 |

variables

| country | year | cases | population |
|-------------|------|-------|------------|
| Afghanistan | 1999 | 75 | 19987071 |
| Afghanistan | 2000 | 866 | 20595360 |
| Brazil | 1999 | 3737 | 17206362 |
| Brazil | 2000 | 8488 | 174604898 |
| China | 1999 | 21258 | 127215272 |
| China | 2000 | 21766 | 128042583 |

observations

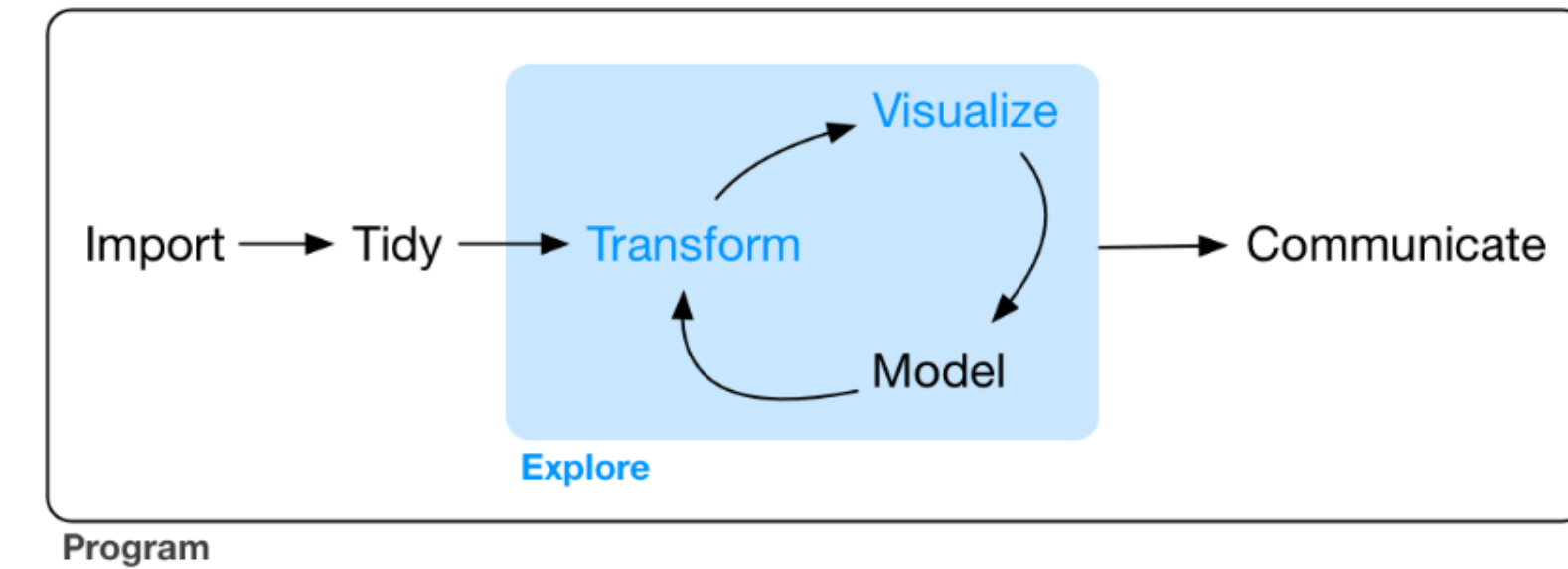
| country | year | cases | population |
|-------------|------|-------|------------|
| Afghanistan | 99 | 75 | 19987071 |
| Afghanistan | 00 | 866 | 20595360 |
| Brazil | 99 | 3737 | 17206362 |
| Brazil | 00 | 8488 | 174604898 |
| China | 99 | 21258 | 127215272 |
| China | 00 | 21766 | 128042583 |

values

- Tidy dataset:
 - Variables are in columns
 - Observations in rows
 - Values in cells

Tidyverse

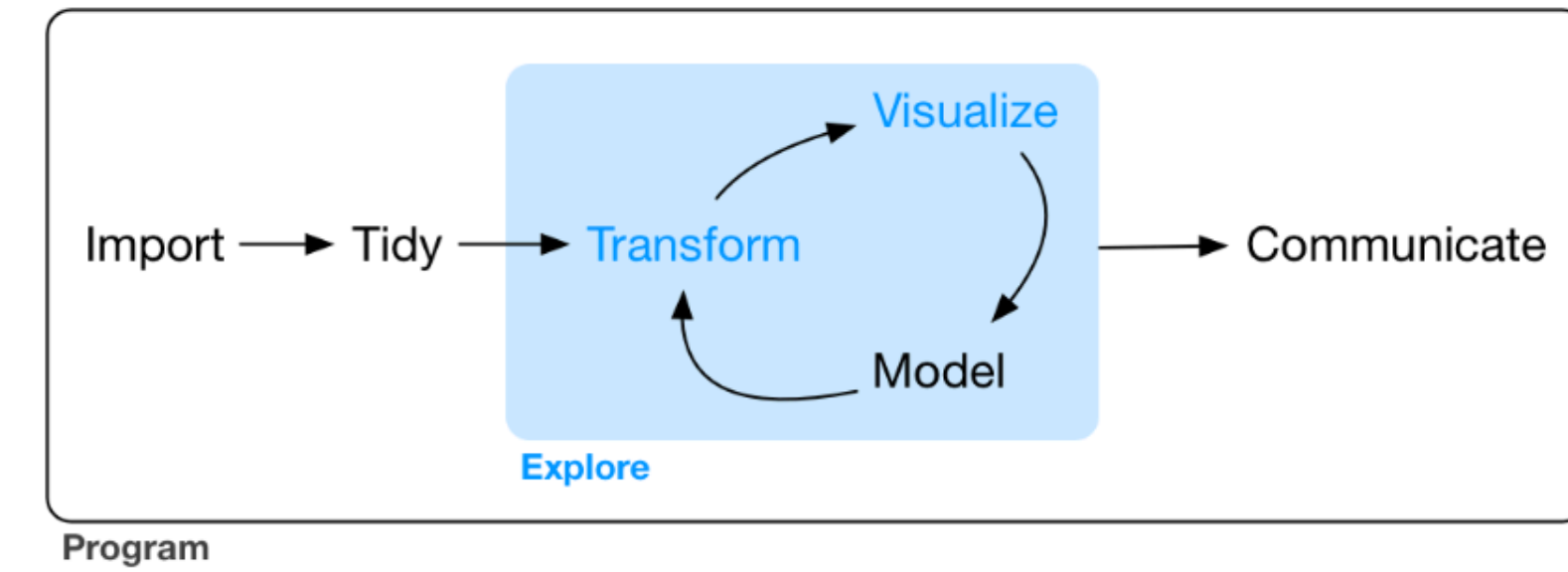
Dplyr — transform



- Data manipulation challenges:
 - Pick observations by their values (`filter()`)
 - Reorder the rows (`arrange()`)
 - Pick variables by their names (`select()`)
 - Create new variables (`mutate()`)
 - Collapse many values to a single summary (`summarize()`)
 - Group rows (`group_by()`)
- All dplyr verbs expect a `data.frame` and produce a new `data.frame`

Dplyr — transform

Select

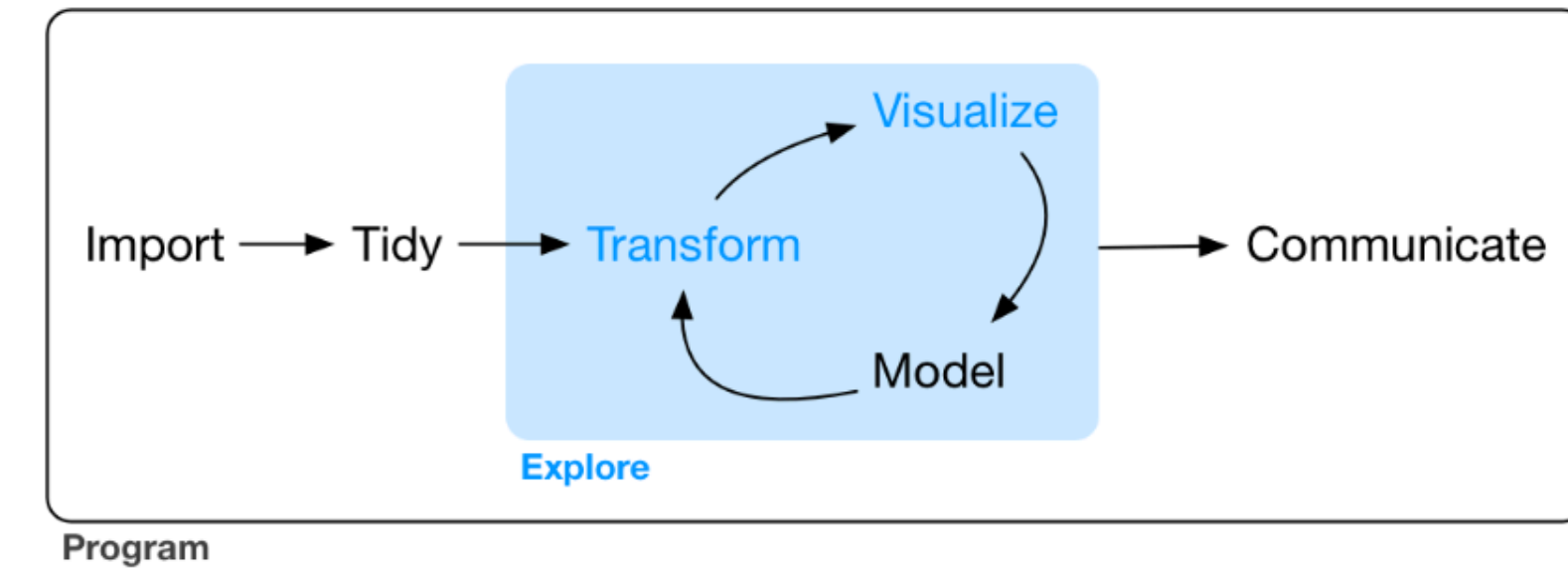


- Select columns with `select()`
 - `mpg %>% select(model,manufacturer,cyl)`
 - `mpg %>% select(model:year)`
 - `mpg %>% select(-(model:year))`
- Helper functions
 - `starts_with("abc")` matches names that begin with “abc”.
 - `ends_with("xyz")` matches names that end with “xyz”.
 - `contains("ijk")` matches names that contain “ijk”.
 - `num_range("x", 1:3)` matches x1, x2, and x3
- `rename(modelo = model)`
- `mpg %>% select(trans, drv, everything())`

Dplyr — transform

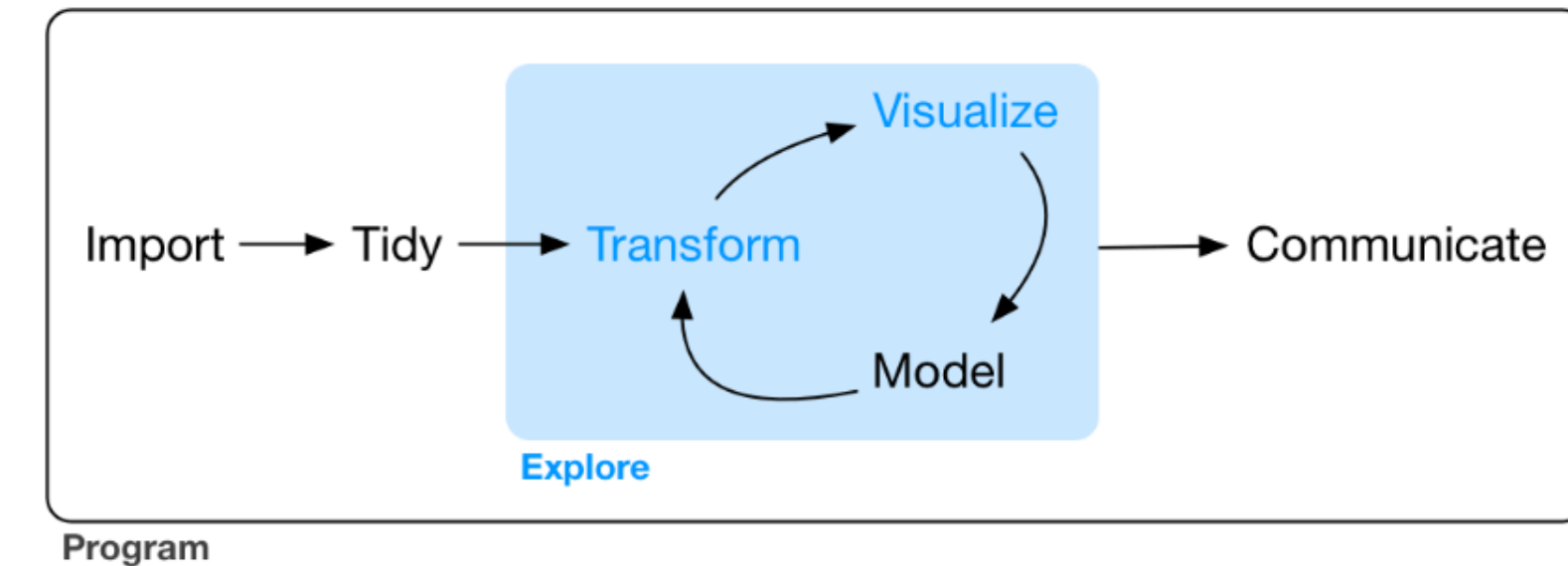
Mutate

- New Variables with mutate()
- mutate() always adds new columns at the end of data frame
 - `mpg %>% mutate(delta=hwy-cty)`
 - `?transmute()`
- Helper functions
 - Arithmetic operators `+`, `-`, `*`, `/`, `^`
 - Logs `log()`, `log2()`, `log10()`
 - Cumulative R provides `cumsum()`, `cumprod()`, `cummin()`, `cummax()`; and dplyr provides `cummean()`
 - Logical comparisons `<`, `<=`, `>`, `>=`, `!=`



Dplyr — transform

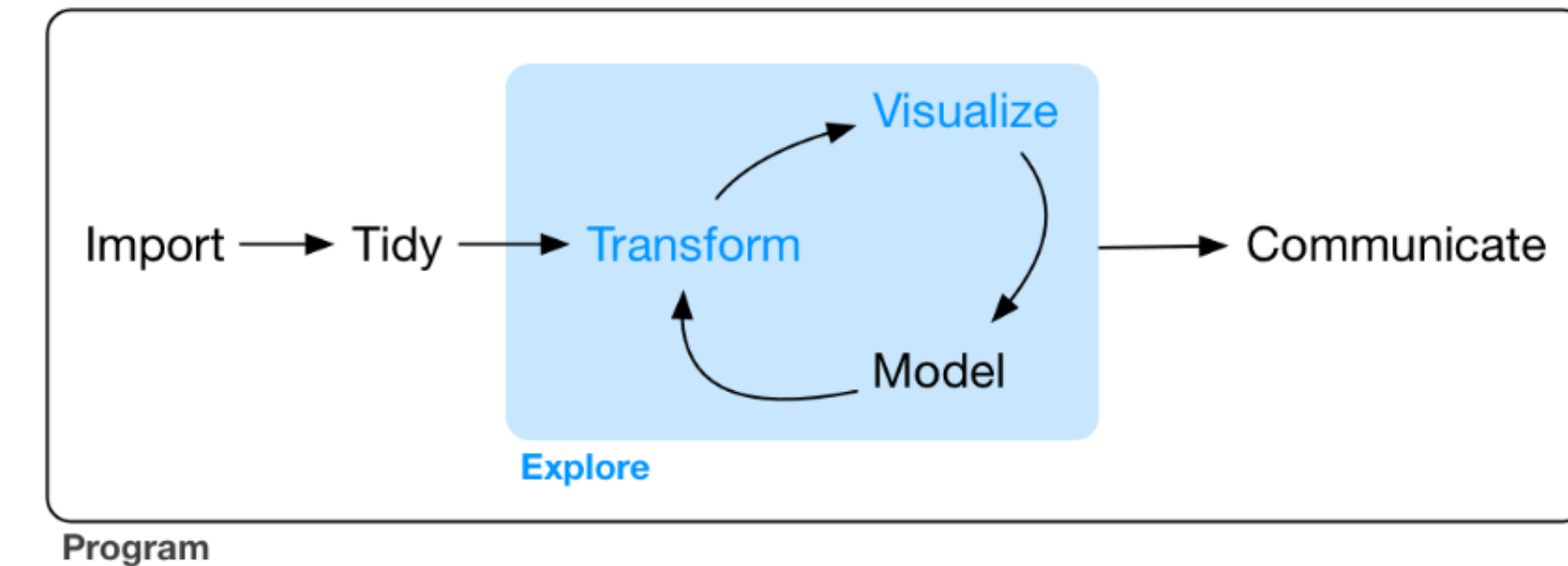
Summarise



- Grouped Summaries with `summarize()`
- collapses a data frame to a single row
 - `mpg %>% summarise(mean(hwy))`
- `summarize()` and `group_by()` a perfect couple
 - `mpg %>% group_by(manufacturer) %>% summarize(m=mean(hwy)) %>% arrange(desc(m))`
- Helper functions
 - Measures of location: `mean(x)`, `median(x)`
 - Measures of spread: `sd(x)`, `IQR(x)`, `mad(x)`
 - Measures of rank: `min(x)`, `quantile(x, 0.25)`, `max(x)`
 - Counts: `n()`, `n_distinct(x)`

Dplyr — transform

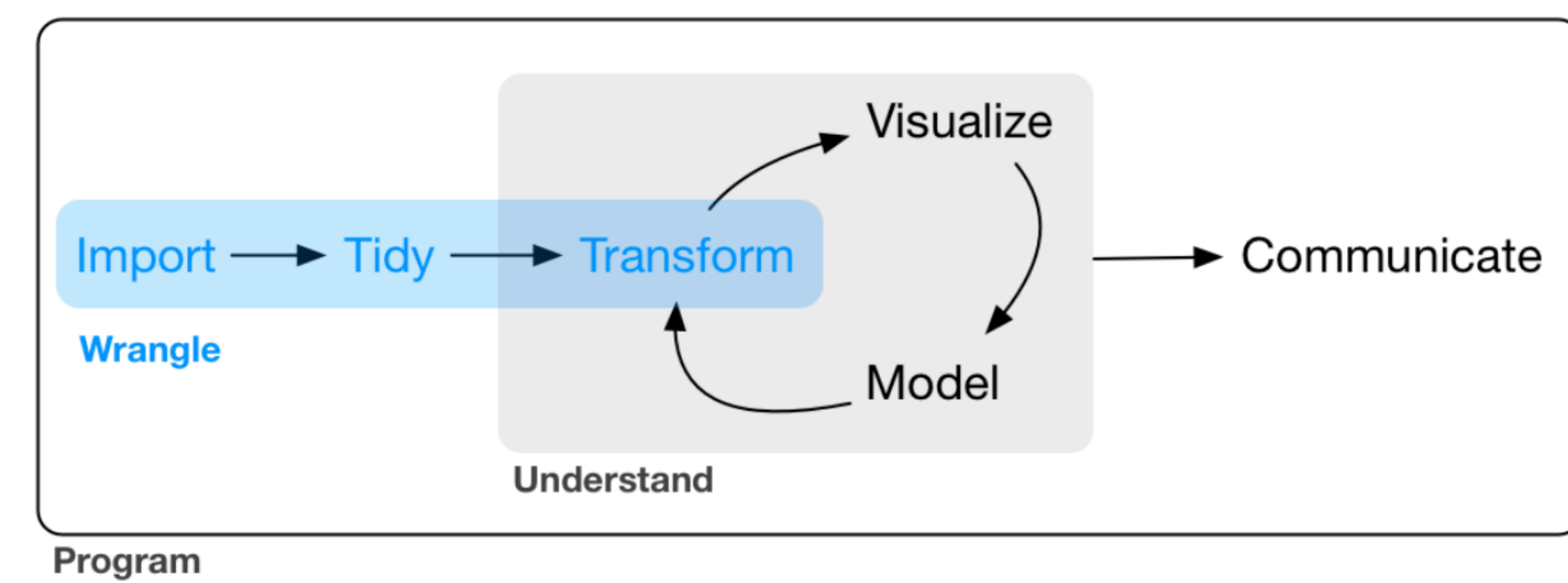
Summarise



- Grouped Summaries with `summarize()`
- collapses a data frame to a single row
 - `mpg %>% summarise(mean(hwy))`
- `summarize()` and `group_by()` a perfect couple
 - `mpg %>% group_by(manufacturer) %>% summarize(m=mean(hwy)) %>% arrange(desc(m))`
- Helper functions
 - Measures of location: `mean(x)`, `median(x)`
 - Measures of spread: `sd(x)`, `IQR(x)`, `mad(x)`
 - Measures of rank: `min(x)`, `quantile(x, 0.25)`, `max(x)`
 - Counts: `n()`, `n_distinct(x)`

Tidyverse

Tidy



- Spreading and Gathering
- Dataset where columns are values of a variable

```
```{r tidy1}
table4a
```
```

A tibble: 3 × 3

| country
<chr> | 1999
<dbl> | 2000
<dbl> |
|------------------|---------------|---------------|
| Afghanistan | 745 | 2666 |
| Brazil | 37737 | 80488 |
| China | 212258 | 213766 |

3 rows

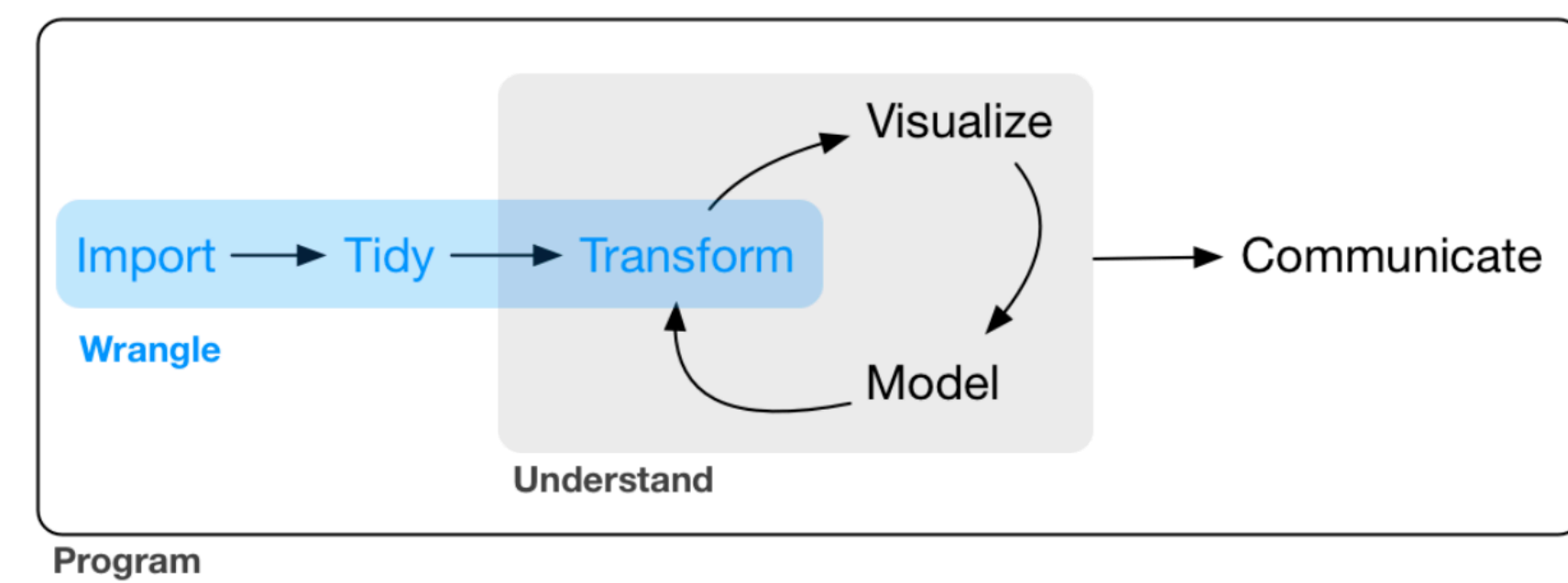
```
```{r tidy1}
table4a %>% gather(`1999`, `2000`, key="year", value="population")
```
```

A tibble: 6 × 3

| country
<chr> | year
<chr> | population
<dbl> |
|------------------|---------------|---------------------|
| Afghanistan | 1999 | 745 |
| Brazil | 1999 | 37737 |
| China | 1999 | 212258 |
| Afghanistan | 2000 | 2666 |
| Brazil | 2000 | 80488 |
| China | 2000 | 213766 |

Tidyverse

Tidy



- Spreading
- Opposite of gathering

| country | year | key | value |
|-------------|------|------------|------------|
| Afghanistan | 1999 | cases | 745 |
| Afghanistan | 1999 | population | 19987071 |
| Afghanistan | 2000 | cases | 2666 |
| Afghanistan | 2000 | population | 20595360 |
| Brazil | 1999 | cases | 37737 |
| Brazil | 1999 | population | 172006362 |
| Brazil | 2000 | cases | 80488 |
| Brazil | 2000 | population | 174504898 |
| China | 1999 | cases | 212258 |
| China | 1999 | population | 1272915272 |
| China | 2000 | cases | 213766 |
| China | 2000 | population | 1280428583 |

table2

```
library(tidyverse)
library(table2)

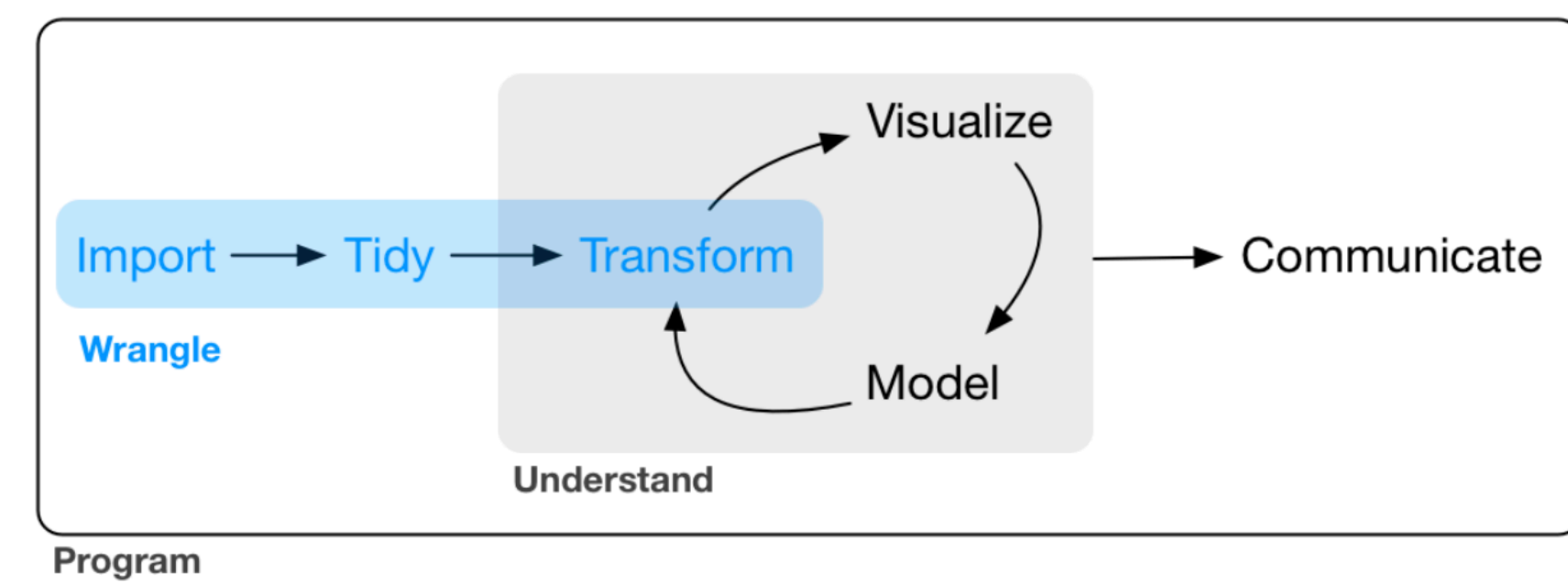
tidy2 <- table2 %>% spread(key=type, value=count)
tidy2
```

A tibble: 6 × 4

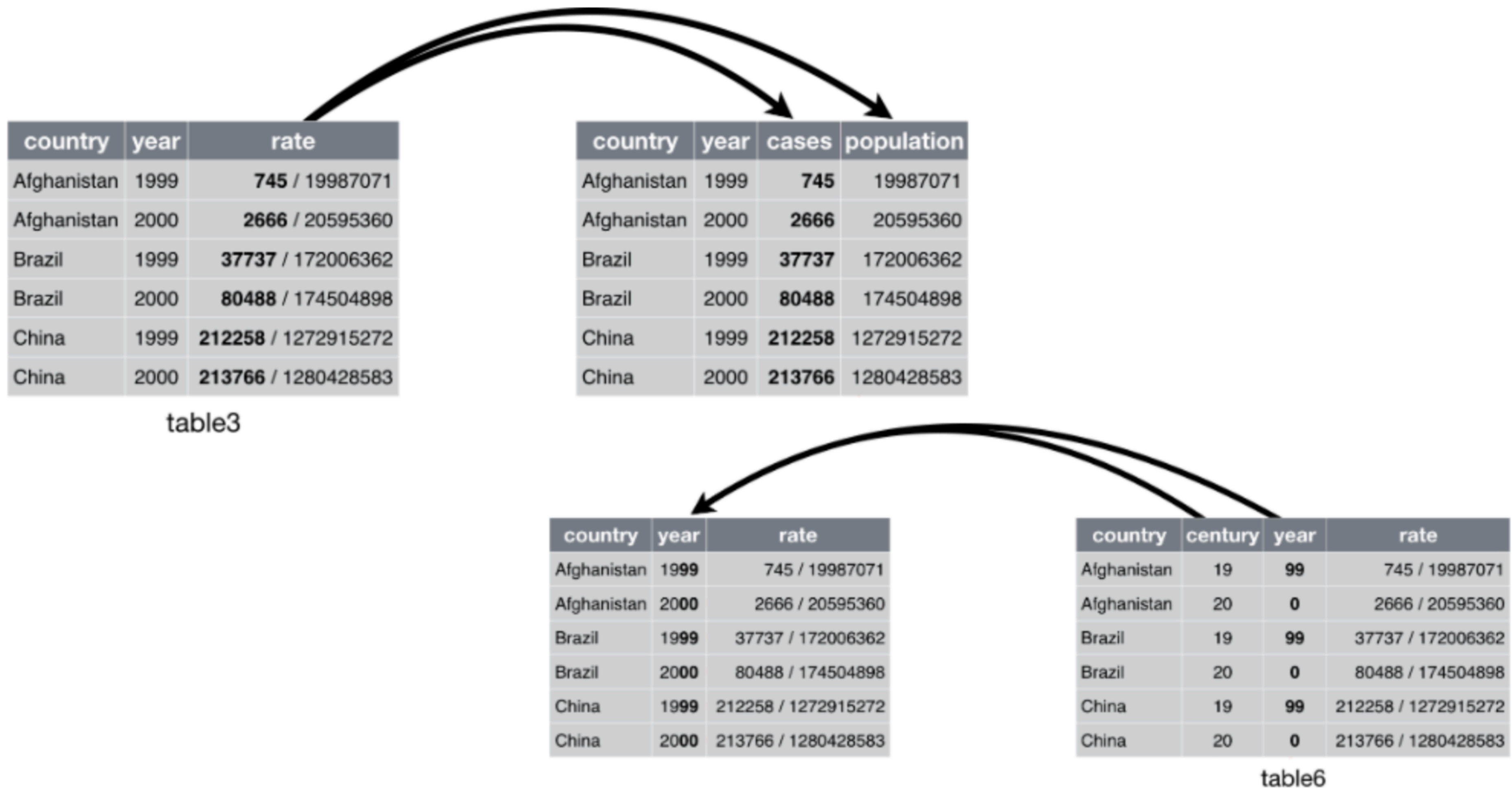
| country
<chr> | year
<dbl> | cases
<dbl> | population
<dbl> |
|------------------|---------------|----------------|---------------------|
| Afghanistan | 1999 | 745 | 19987071 |
| Afghanistan | 2000 | 2666 | 20595360 |
| Brazil | 1999 | 37737 | 172006362 |
| Brazil | 2000 | 80488 | 174504898 |
| China | 1999 | 212258 | 1272915272 |
| China | 2000 | 213766 | 1280428583 |

Tidyverse

Tidy



- Separate() and unite()



Tidyverse

Reorganizando valores

- `pivot_longer()` y `pivot_wider()`

`pivot_longer`(data, cols, names_to = "name",
values_to = "value", values_drop_na = FALSE)

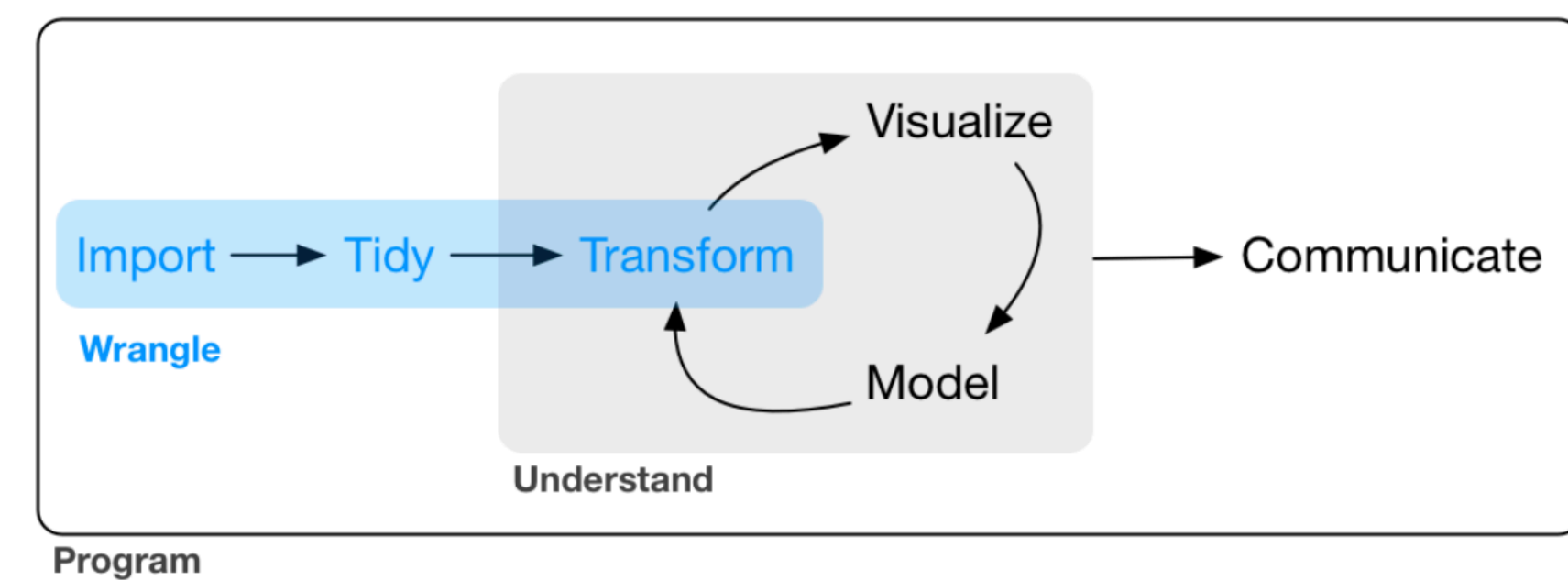
table4a

| country | 1999 | 2000 |
|---------|------|------|
| A | 0.7K | 2K |
| B | 37K | 80K |
| C | 212K | 213K |

→

| country | year | cases |
|---------|------|-------|
| A | 1999 | 0.7K |
| B | 1999 | 37K |
| C | 1999 | 212K |
| A | 2000 | 2K |
| B | 2000 | 80K |
| C | 2000 | 213K |

`pivot_longer`(table4a, cols = 2:3, names_to = "year",
values_to = "cases")



`pivot_wider`(data, names_from = "name",
values_from = "value")

table2

| country | year | type | count |
|---------|------|-------|-------|
| A | 1999 | cases | 0.7K |
| A | 1999 | pop | 19M |
| A | 2000 | cases | 2K |
| A | 2000 | pop | 20M |
| B | 1999 | cases | 37K |
| B | 1999 | pop | 172M |
| B | 2000 | cases | 80K |
| B | 2000 | pop | 174M |
| C | 1999 | cases | 212K |
| C | 1999 | pop | 1T |
| C | 2000 | cases | 213K |
| C | 2000 | pop | 1T |

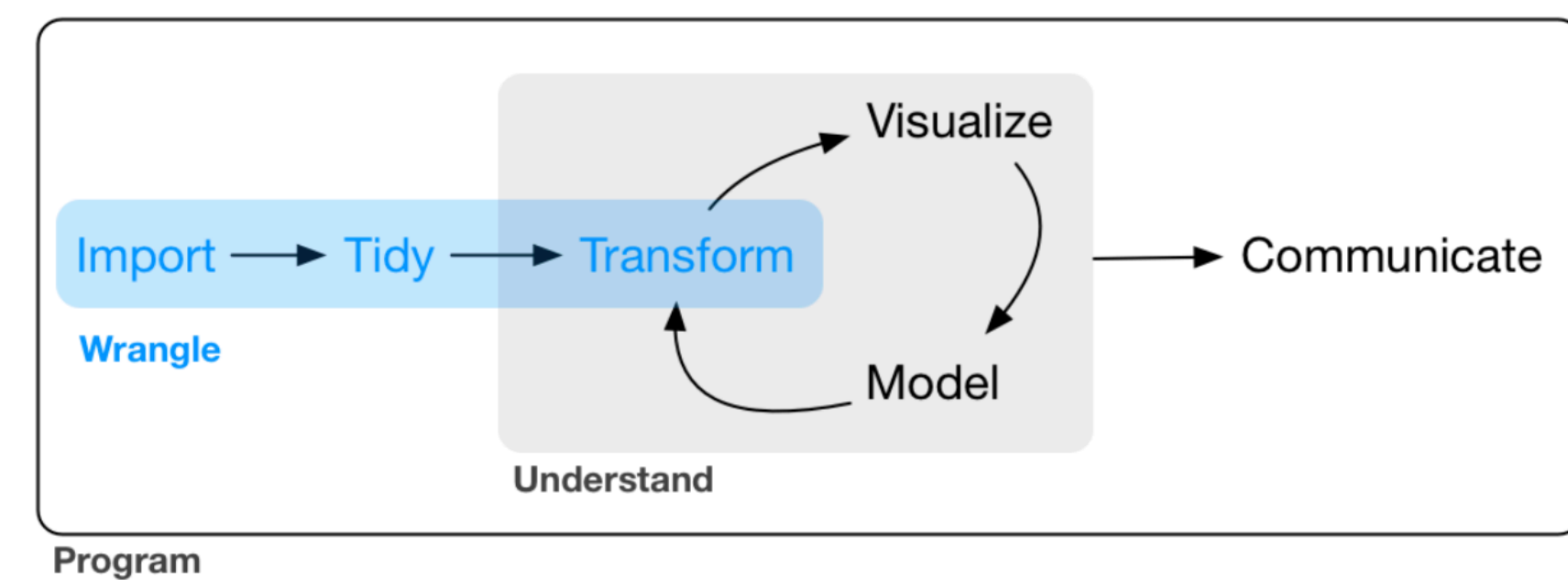
→

| country | year | cases | pop |
|---------|------|-------|------|
| A | 1999 | 0.7K | 19M |
| A | 2000 | 2K | 20M |
| B | 1999 | 37K | 172M |
| B | 2000 | 80K | 174M |
| C | 1999 | 212K | 1T |
| C | 2000 | 213K | 1T |

`pivot_wider`(table2, names_from = type,
values_from = count)

Tidyverse

Varias funciones



X

| x1 | x2 |
|----|----|
| A | 1 |
| B | NA |
| C | NA |
| D | 3 |
| E | NA |

drop_na(data, ...) Drop rows containing NA's in ... columns.

`drop_na(x, x2)`

X

| x1 | x2 |
|----|----|
| A | 1 |
| B | NA |
| C | NA |
| D | 3 |
| E | NA |

fill(data, ..., .direction = "down") Fill in NA's in ... columns using the next or previous value.

`fill(x, x2)`

X

| x1 | x2 |
|----|----|
| A | 1 |
| B | NA |
| C | NA |
| D | 3 |
| E | NA |

replace_na(data, replace) Specify a value to replace NA in selected columns.

`replace_na(x, list(x2 = 2))`

X

| A | B | C |
|---|---|---|
| a | t | 1 |
| b | u | 2 |
| c | v | 3 |

+

y

| E | F | G |
|---|---|---|
| a | t | 3 |
| b | u | 2 |
| d | w | 1 |

=

| A | B | C | E | F | G |
|---|---|---|---|---|---|
| a | t | 1 | a | t | 3 |
| b | u | 2 | b | u | 2 |
| c | v | 3 | d | w | 1 |

bind_cols(..., .name_repair) Returns tables placed side by side as a single table. Column lengths must be equal. Columns will NOT be matched by id (to do that look at Relational Data below), so be sure to check that both tables are ordered the way you want before binding.

X

| A | B | C |
|---|---|---|
| a | t | 1 |
| b | u | 2 |

+

y

| A | B | C |
|---|---|---|
| c | v | 3 |
| d | w | 4 |

| DF | A | B | C |
|----|---|---|---|
| x | a | t | 1 |
| x | b | u | 2 |
| y | c | v | 3 |
| y | d | w | 4 |

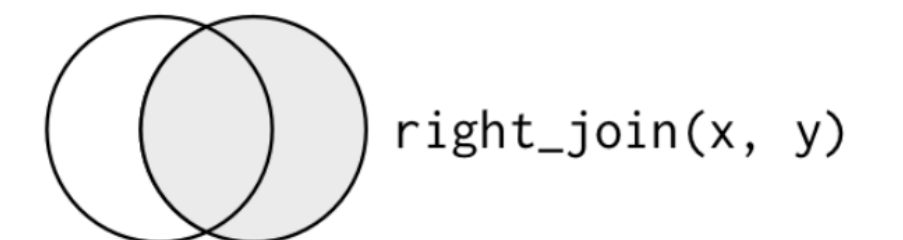
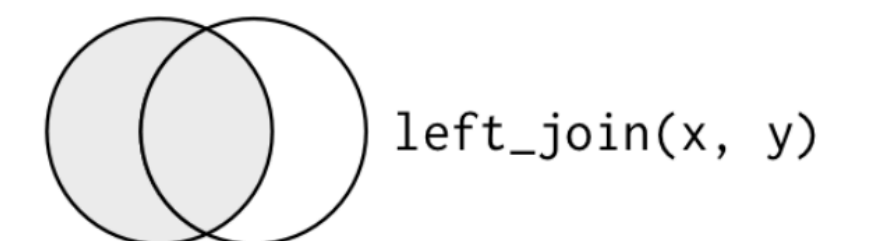
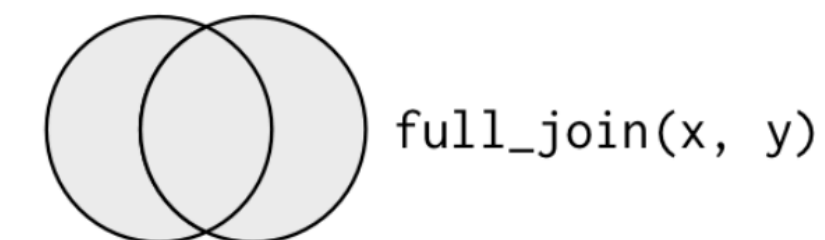
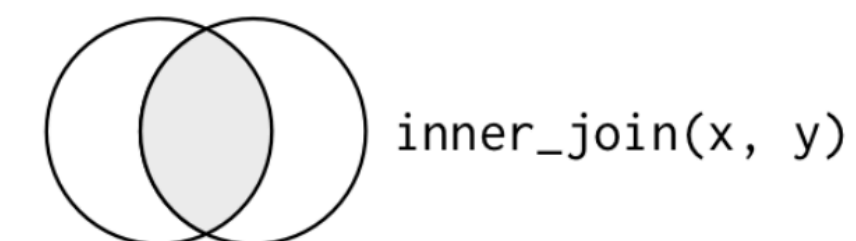
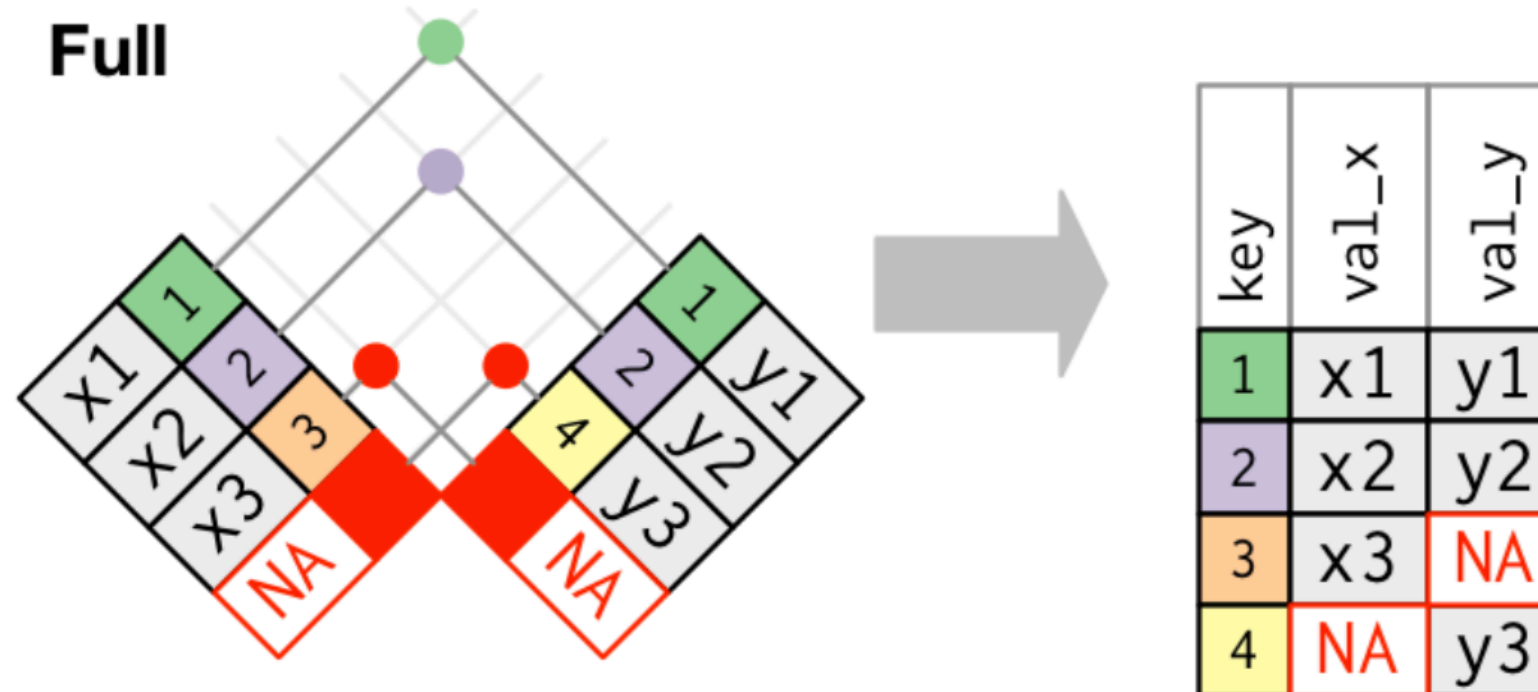
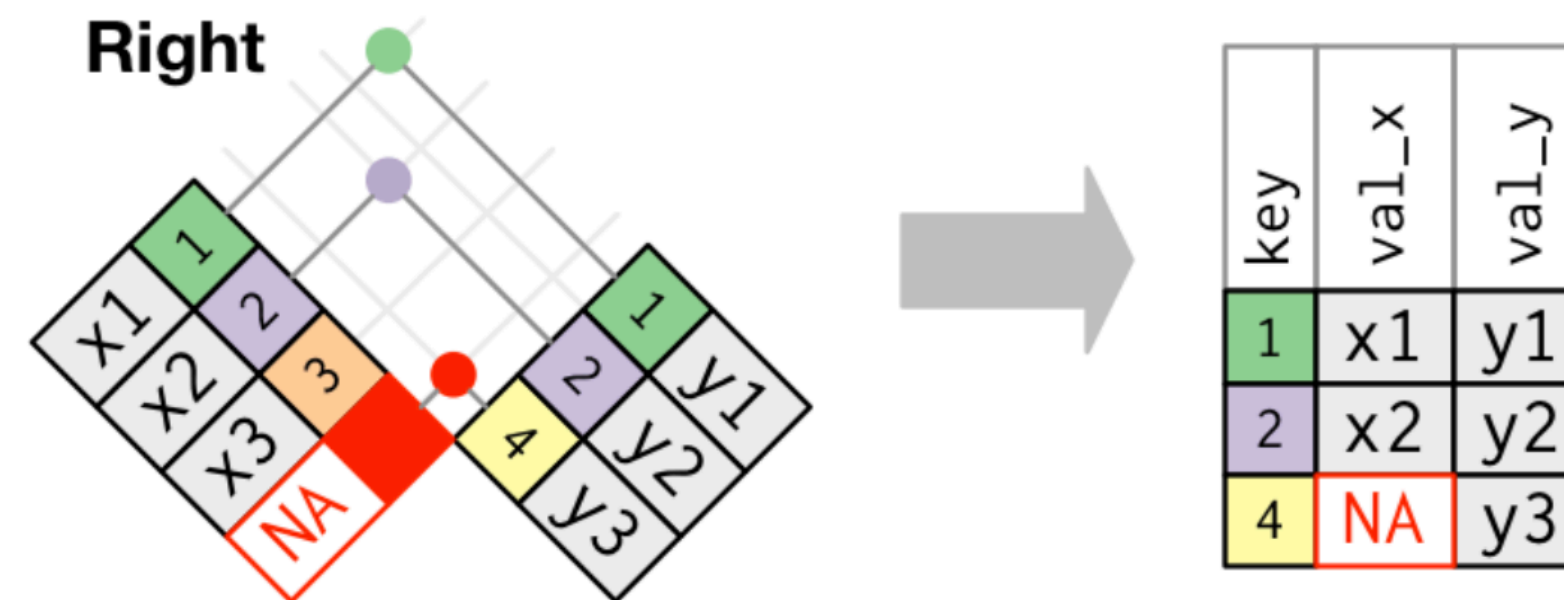
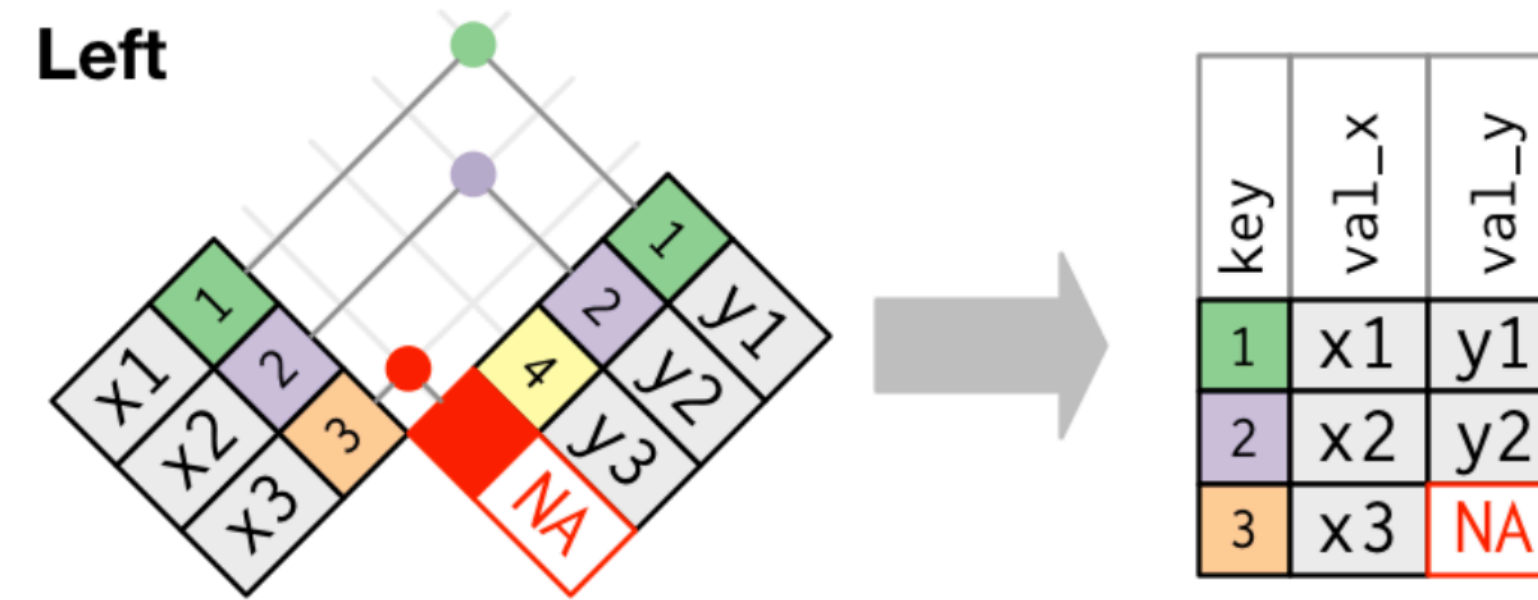
bind_rows(..., .id = NULL) Returns tables one on top of the other as a single table. Set .id to a column name to add a column of the original table names (as pictured).

Tidyverse

Joins

- Merging two data frames by a key.
- `left_join(x,y, by= "key")`
- `left_join(x,y,by=c("key1"="key2"))`

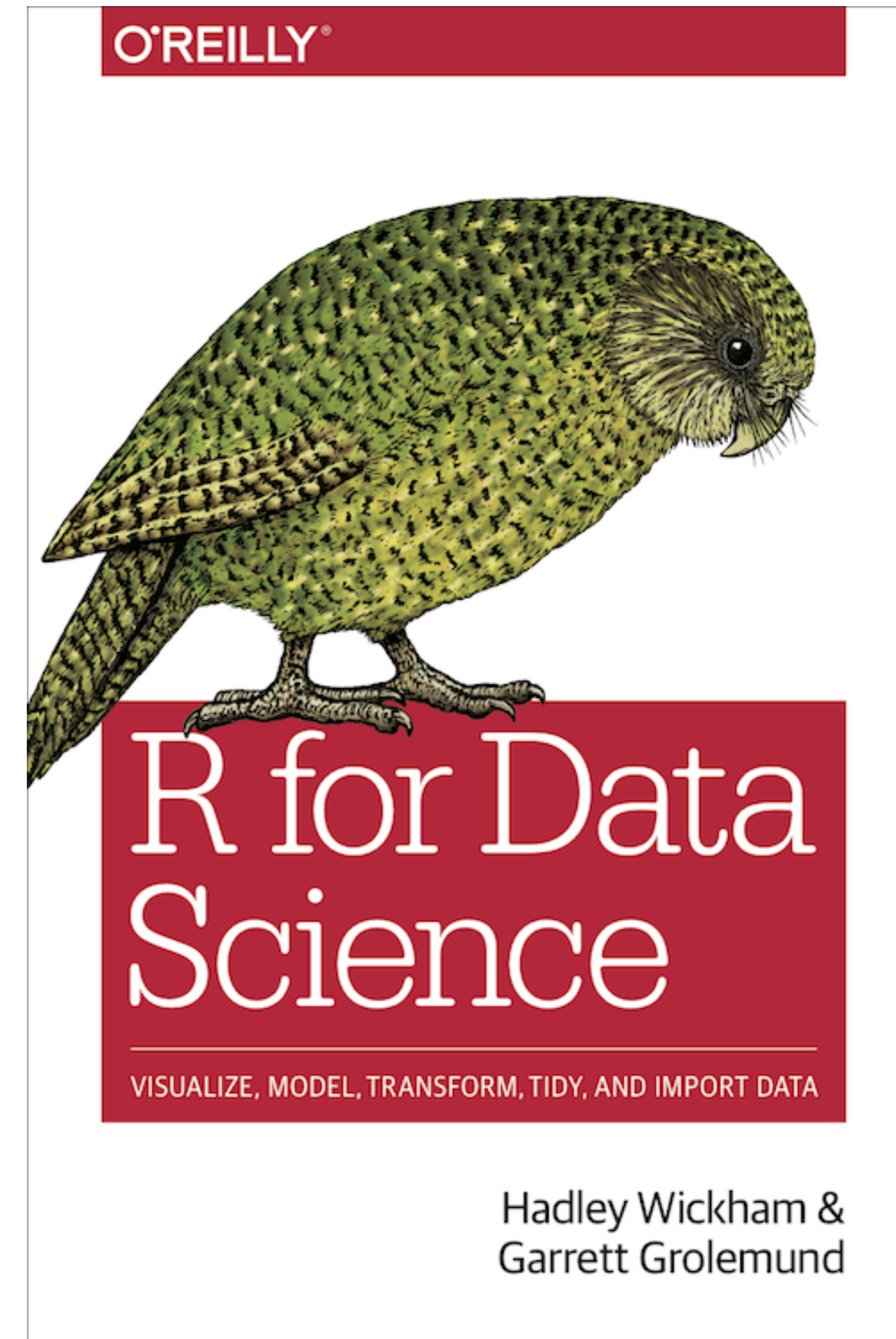
| dplyr | merge |
|-------------------------------|--|
| <code>inner_join(x, y)</code> | <code>merge(x, y)</code> |
| <code>left_join(x, y)</code> | <code>merge(x, y, all.x = TRUE)</code> |
| <code>right_join(x, y)</code> | <code>merge(x, y, all.y = TRUE),</code> |
| <code>full_join(x, y)</code> | <code>merge(x, y, all.x = TRUE, all.y = TRUE)</code> |



R4ds

Material adicional

- <https://es.r4ds.hadley.nz/>



**¿Consultas o
comentarios?**